Multi-Robot Optimal Path planning Using Nature Inspired Algorithms

Abstract

Nature has inspired many algorithms to solve complex problems. Understanding how and why these models of nature work not only leads to new understandings of nature but also to understanding the deeper connections between known algorithms. Here we show that network properties underlie and define a whole family of nature-inspired algorithms. In particular, the search is based on networks defined by neighboring tasks in the landscape (real or virtual), and the phase transition mediates local and global searches. His three computing paradigms (two-stage evolution, evolutionary dynamics, and general local search engines) provide a theoretical basis for understanding how nature-inspired algorithms work. Offer. Several algorithms provide useful examples, notably genetic algorithms, ant colony optimization, and simulated annealing. At a time when technology moves and improvises quickly, according to Robotics, it plays an important role in justifying this pace. The contribution of service robots can be seen in almost all fields. Robots are used to improve the efficiency and accuracy of any application or environment. The demand for adoption is increasing day by day. Now, robots and robotics have moved beyond marginal acceptance in some defined fields and are approaching wider acceptance. In this study, we propose an optimized path planning and control technique for robots. The first step aims to optimize the path planning of the robot by detecting obstacles in the workspace. This is achieved by implementing ACO, which detects surrounding obstacles in time. This feature includes a robot planning algorithm that helped improve robot planning algorithms using the PSO algorithm for path length and time to improve robot coordination and planning techniques. The most tedious

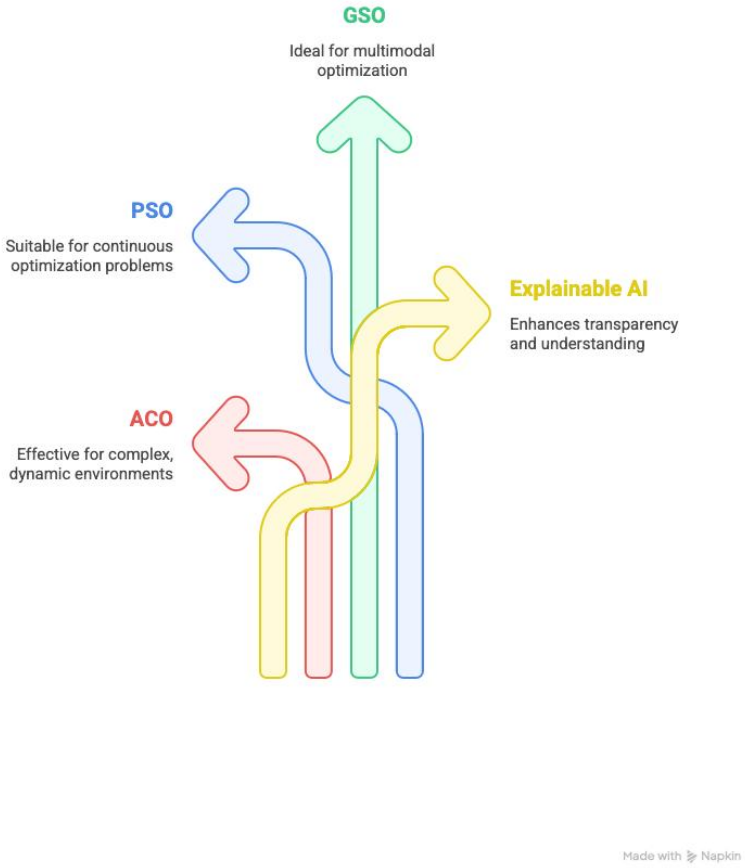

Figure 1: Graphical Abstract

part of any algorithm is to achieve a free trajectory, and the most time-consuming mechanism for algorithms to achieve this is when obstacles are encountered in the path, those obstacles are removed. The proposed approach successfully solves this problem and is presented. The next stage of work focused on optimizing the path length and path time by modifying the GSO algorithm by adopting the tournament selection method to select the nodes of the robot path. The proposed approach solved the minimal problem of the cuckoo search algorithm in finding the optimized path of the robot. The APGO search algorithm modified by the tournament selection method performed better with both parameters and achieved good results. The same was presented in the second stage of the work.

Contents

List of Figures

List of Tables

List of Algorithms

0.1 Part1

Chapter 1

Introduction

1.1 Introduction

A robot is an artificial agent, which acts instead of a person for doing things it is designed for. Robots are usually machines controlled by a computer program or electronic circuitry. They may be directly controlled by humans. Most robots are utilized for specific jobs, and they are available in many forms [1]. Robots have many uses. Many factories use robots to achieve accuracy and efficiency in their work. The robots are of two types, Fixed and Mobile. The fixed-mobile is one which are mounted once in a place, it will perform the task from that place without shifting but can have many degrees of freedom. Robots utilised in the factories are mostly fixed-base robots.Mobile robots are those which can change their place as per the requirements and do have many degrees of freedom. Now a day, the use of mobile robots in industries is also increasing. The big advantage of mobile robots is their inherent flexibility [2]. Irrespective of the surrounding condition, mobile robots can be designed and utilized for a variety of applications. Some common examples are in the field of surveillance, military, and space exploration. In this work, our preferred device for research is the Mobile Robot, here we are initially using two robots for analysis [3].

1.2 Path Planning

A fundamental requirement for independent mobile robots in most of their applications is the skill to navigate from origin to a given goal [4]. The mobile robot must be able to generate a collision-free path that connects the point of origin and the given goal.

Robot direction-finding method can be divided into three categories either local navigation, global navigation, and hybrid method. In global navigation, method robots know the environment and path are generated to avoid the obstacle [5]. But in the case of the local navigation method, the robot collects the local information based on the sensors. When the environment is changed, then the cost will be more in the global method, because the supply of new maps is difficult [6]. But in the case of local navigation, the input of the map is not required so when the environment is changed it will work properly.

Multi-Robot Path planning is a very important and challenging problem in the multi-robot system. It guides the robots to work together cooperatively without colliding with each other and obstacle [7].

Multi-robot path planning can be defined as follows:

Suppose there is n robot of k dimension, and each robot has an initial configuration that will define the starting and ending point for the robot. The objective is to determine the path for each robot that will avoid the obstacle as well as the other robot. So here multi-robot is enabled to navigate collaboratively to achieve the goal. In multi-robot collaborative path planning the objectives is to minimize the total path, minimize the total path length, and minimize the energy to reach the goal [8].

1.3 Evolution of Path Planning Problem

For humans, getting from one location to the next is a simple undertaking. One makes a close decision about how to proceed. Such a straightforward operation is

a significant difficulty for robots. Path planning is a crucial challenge in robotic applications. Finding a sensible route for a robot to travel from an initial location to a destination point, whether it be a cleaning robot [9], a mechanical arm, or a mysteriously flying machine is indeed a standard challenge. Finding a route from a starting place to the desired position is the issue. Based on the modeling framework, the sort of robots, the purpose, etc. This problem was approached in many ways in this study.

A path optimization method that is productive is necessary for safe and reliable mobile robot navigation because the integrity of the successful approach has a significant impact on industrial applications. As it affects other parameters like computation time and resource usage [10], decreasing the distance traveled is usually the primary goal of the mapping process.

In this survey, we have studied the past 4 decades of research articles to understand the evolution of path-planning algorithms. Initially, researchers started to work with Classical and Hybrid approaches. Path planning issues are frequently addressed with approaches from the conventional category, including Cell Decomposition (CD), Potential Field (PF), Road Map, and Subgoal Network. These techniques are understood to be computationally costly and are shown to be less effective in managing unfamiliar [11], imperfectly known, or dynamic settings in their basic formulation. To build a viable path among the source and target places, they are also frequently found to rely on thorough previous information about the environment. These problems can be resolved by meta-heuristic methods since they are adept at managing unknowable or imperfectly known settings.

Each repetition of these procedures generates a series of interim paths that get them one algorithmic step closer to the final location. In contrast to conventional methods, which generate a global path based on previous information (such as an environment map) and then implement it, meta-heuristic techniques produce a subpopulation of potential movements in each loop [12]. Following that, the optimal path will be selected and carried out based on their fitness.

Some researchers viewed meta-heuristic-based techniques for path planning as being based on techniques like Genetic Algorithms (GA), Neural Networks (NN), Particle Swarm Optimization (PSO), and Ant Colony Optimization (ACO). To improve mobile robots' skills to deal with stationary and moving obstacles, several experts combined neural network models and reinforcement learning [13]. These problems can be resolved by meta-heuristic methods simply because they are adept at managing unknowable or imperfectly known settings. Each repetition of these procedures generates a series of provisional paths that get them one algorithmic step closer to the final position.

Multi-robot path planning can be defined as follows: Suppose there is X robot of D dimension, and each robot has a preliminary configuration that will define the starting and ending point for every robot. The aim is to determine the path for each robot independently that will avoid the obstacle as well as the other robot(i.e. static and dynamic hurdles) [14]. So here multi-robot is allowing to navigate collaboratively to achieve the goal. In multi-robot collaboratively path planning the objectives is to minimize the total possible path, minimize the path length, minimize time and to minimize the energy to reach the goal. Following are the classification of the path panning shown in fig 3.1.

1.4 Nature Inspired Algorithms

Nature-Inspired Optimization Algorithms provide a systematic introduction to all major nature-inspired algorithms for optimization. It includes particle swarm optimization, ant and bee algorithms [15], simulated annealing, cuckoo search, firefly algorithm, bat algorithm, flower algorithm, harmony search, algorithm analysis, constraint handling, hybrid methods, parameter tuning, and control, as well as multi-objective optimization. The different Types of nature-inspired algorithms:

1. Ant Colony Optimization Algorithm (ACO);
2. Particle swarm optimization (PSO);
3. Glow worm swarm optimization (GSO);

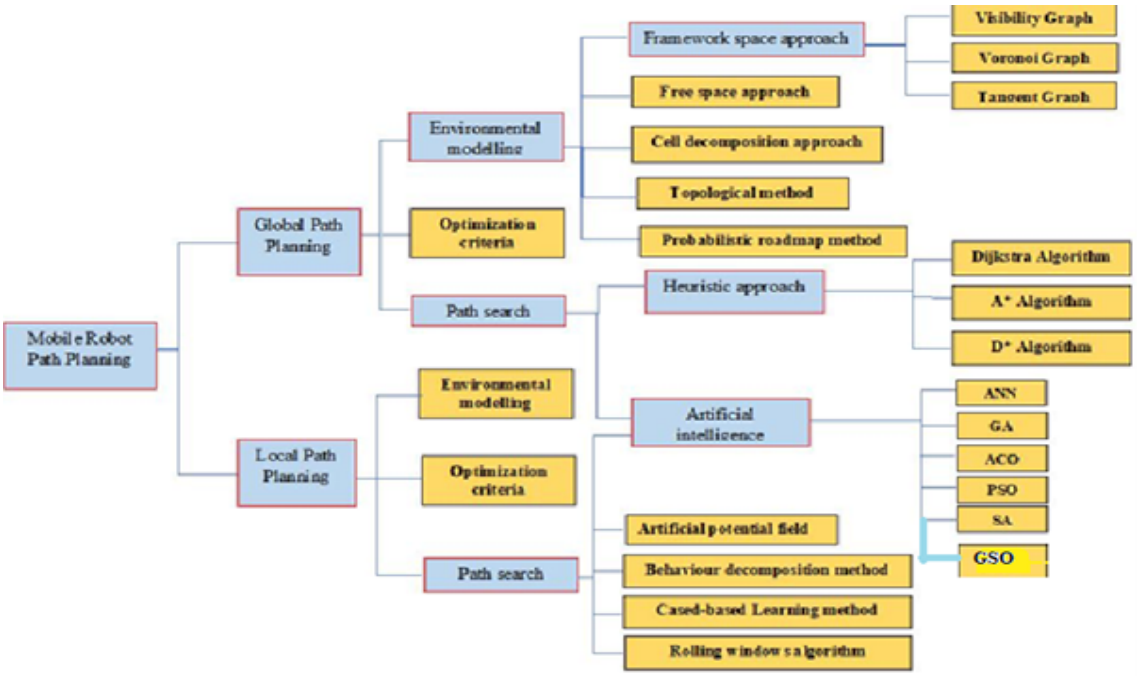

Figure 1.1: Approaches for solving path planning problem

4. Artificial Immune System (AIS) etc..

1.5 Multi Robot System

Cell robots are computerized or human-managed dealers that move in pre-designed environment. These are mobile delegate that, as the name suggests, move from one physical location to another. Mobile robots are tending to a amazing deal of cutting-edge studies.. Mobile robots are also found in industrial, military, along with hospitals, industries, museums, navy, motors, educate stations, or even leisure and safety houses [16]. They also appear as consumables or objects for entertainment purposes or to perform certain duties including vacuuming, gardening and a few other not unusual household obligations.

A system consisting of several interactive intelligent robots. A multi-robot system approaches the problem of AI research, i.e., design, reasoning, search and machine learning. Recently, multi-robot systems have emerged as an affordable and robust way to address many overwhelming challenges. Such applications include area

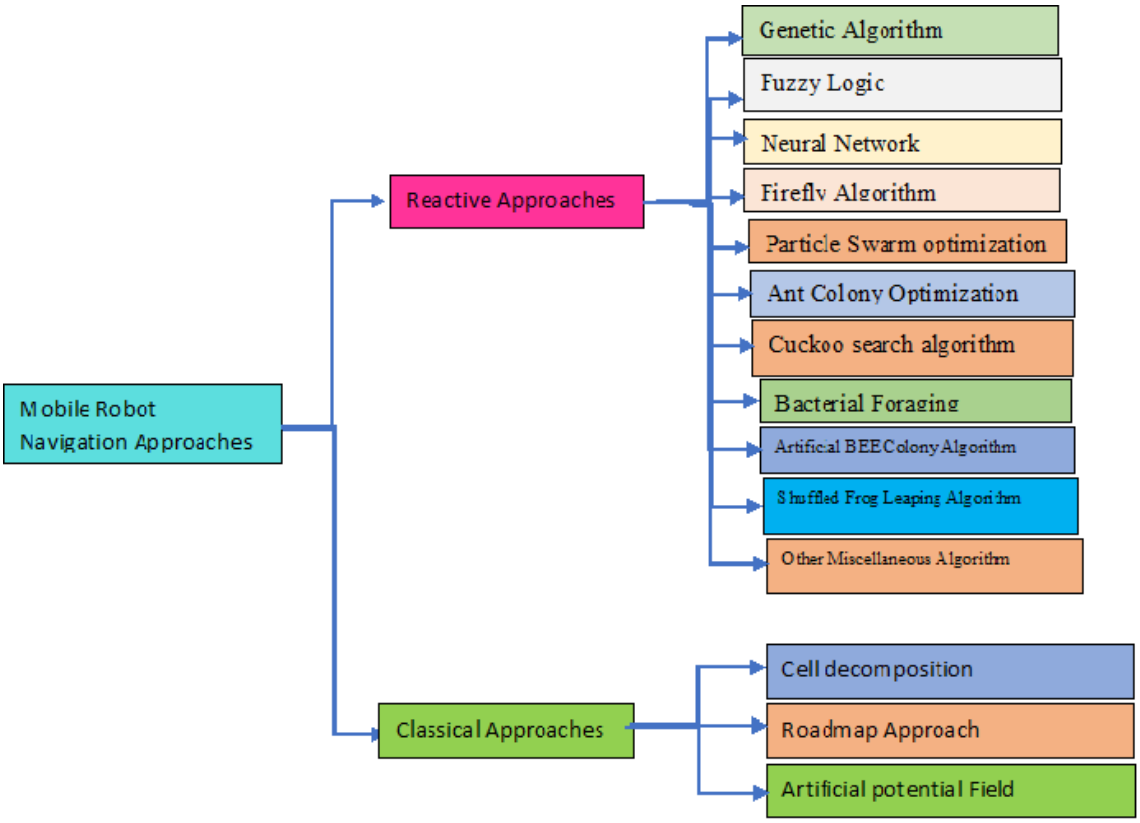

Figure 1.2: mobile navigational approachs

search, surveillance, target tracking, rescue operations, environmental monitoring, reconnaissance, cooperative building and manipulation, and more. All these applications require covering the region of interest with robotic sensors or end-effectors for different purposes [17]. The use of multiple robots enables redundancy and offers opportunities to increase efficiency. There are certain reasons for choosing a multi-robot system:

1. Some quests require a team.
2. You can complete tasks faster.
3. Strengthening force.
4. Multiple bots allow for more versatile and creative solutions.
5. Can handle incomplete communication.

Compared to other distributed systems, multi-robot systems are more complex because they have to deal with a real environment that is more difficult to simulate because it is unpredictable, dynamic, noisy, etc.

1.6 Motivation

Multi-Robot Optimal Path Planning Using Nature Inspired Algorithm is being decided on as the extensive scene of research where path planning and coordination problem is taken into consideration to carry out the task similarly [18].

1.7 Objectives

The primary objectives of the thesis are as follows:

- To broaden a technique for locating collision-free paths with reduced course periods and computation time for Robots.
- To broaden the way for early detecting the whole periphery of the perfect edge of contours for smooth robotic navigation in uneven geographical planes.
- Develop a robust method to coordinate loosely coupled robots by improving the coordination mechanism in a multi-robot scenario [18].

1.8 Hypothesis

- Null Hypothesis (H0): There is no significant improvement in robot path planning performance when utilizing a hybrid algorithm that combines ACO, PSO, and GSO compared to using each algorithm individually.
- Alternative Hypothesis (H1): The hybrid algorithm combining ACO, PSO, and GSO will result in improved robot path planning performance compared to using each algorithm individually

1.9 Thesis organization

The research carried out is aimed to improve the performance of robot in regard with the path planning and coordination. The concept of clubbing the robots with cloud platforms and to increase the performance of parameters is the ultimate goal of the thesis. The work is initially carried out for single robot environment and then

shifted to the multi robot scenario and finally into the cloud based robots scenario. The primary contributions of the thesis are as follows:

Chapter 2 presents an in depth details of the survey carried out for robot's path planning problem. Several algorithms which are well known and aggressively adopted by many researchers are discussed in detail. All the pros and cons of each work mentioned is described in detail. A rigorous survey is carried out for each objective proposed in the work and the analysis of all those work is systematically presented in this section. Like which particular algorithm is deployed for single robot or multi robot scenario and which particular parameters are considered to validate the work. All the issues related to robot path planning environment is discussed in detail in this section.

Chapter 3 represents the evolution of path planning problem and the algorithms that include traditional, heuristic and meta heuristic optimization techniques. They are capable of handling complex and non-linear constraints, they can find globally optimal solutions, and they are relatively easy to implement and can be applied to a wide range of problems. These algorithms are a promising direction for mobile robot path planning. By leveraging the principles of natural systems, these algorithms can provide effective and efficient solutions for a wide range of problems in the field of robotics.

Chapter 4 presents the hybrid approach of nature inspired algorithms. Nature-inspired algorithms such as genetic algorithms, particle swarm optimization, and ant colony optimization have been widely used in the field of mobile robot path planning. These algorithms are inspired by the behaviour of natural systems, such as the behaviour of animals in a colony, the movement of particles in a fluid, and the process of natural selection in biology. These algorithms have several advantages over traditional optimization techniques when it comes to mobile robot path

planning. APGO algorithm is proposed by the fusion of PSO, ACO, and GSO to get the benefits of all the three approaches.

Chapter 5 This chapter investigates the use of iterative refinement in multi-agent path finding (MAPF), a path planning problem for multiple robots. We analyze and compare several path planning algorithms, including ICBS, HCA, PIBT, CBS, and WHCA, to identify the most effective approach to finding optimal paths for multiple agents. Using these algorithms, we develop a sub-optimal MAPF solver that quickly generates initial solutions, which we then refine through iterative selection of a subset of agents and application of an optimal MAPF solver. Our study provides valuable information on the effectiveness of iterative refinement in MAPF and can help researchers identify the most efficient approach to this problem. These findings have practical applications in areas such as robotics, logistics, and transportation, where efficient path planning for multiple agents is essential to achieve optimal performance.

Chapter 6 This chapter presents the conclusion and summarizing of the overall work presented. Once concluding all the objectives are accomplished it also analyzes the future course of action which could be performed as an extension of this work. All the possible course of work with respect to each objective is explained in this chapter followed by the conclusion.

Chapter 2

Literature Survey

2.1 Background

In a physical environment, cell robots can tour their surroundings to perform diverse tasks. Due to their more desirable mobility, cell robots find applications in many locations which include hospitals, industries, museums, military, automotive, railway stations, and even houses for entertainment and surveillance purposes. Robotic navigation is an essential hassle in robotics. Navigation of a mobile robot includes the capability to get a free route from the place to begin to the destination factor even by avoiding boundaries. further, the chosen route ought to be optimized (i.e, it ought to have the smallest feasible distance and the minimum variety of revolutions) so that the robot makes use of as little electricity and time as viable in shifting from its starting point to its destination.

Cellular robots use a diffusion of locomotion mechanisms, inclusive of wheels, legs, and tracks. Thanks to their efficiency and adaptability, wheeled cellular robots are regularly becoming necessary in an enterprise as a method of shipping, manipulating, and paintings. similarly, cell robots are useful to intervene in two damaging environments to perform tasks which include terminal detection, nuclear waste control, nuclear reactor cleaning, surveillance, etc. in addition, cellular robots can act as a check platform for far-off work. Assessments for flexible use. This thesis offers the layout and development of clever controllers for course-making plans and navi-

gation of numerous cell robots in an acknowledged and unknown environment such as various boundaries.

Navigation methods for mobile robots may be classified into worldwide direction-making plans and neighborhood path-making plans. In global path planning, the robot has completed earlier records about the form, region, course, or even arrangement of barriers inside the environment. It makes use of some optimization algorithms to minimize the cost function and might handiest work in a static environment. However, in a local direction making plans, the robot uses some reactive techniques such as an artificial neural network to come across the environment primarily based on the data amassed from various sensors and plan the path online.

Mobile robots are automatic or human-controlled agents that move in their environment. These are mobile agents that, as the name suggests, move from one physical location to another. Mobile bots are driving a lot of current research. Mobile robots are also found in industrial, military, and security environments. They are also used as consumer goods or objects, for entertainment purposes or to perform certain tasks such as vacuuming, gardening, and some other common household tasks. A system consisting of several interactive intelligent robots. A multi-robot system approaches the problem of artificial intelligence research, i.e. design, reasoning, search, and machine learning. Recently, multi-robot systems have emerged as an affordable and robust way to address many overwhelming challenges. Such applications include area search, surveillance, target tracking, rescue operations, environmental monitoring, reconnaissance, cooperative building and manipulation, and more. All these applications require that the region of interest be covered by the sensors or end effectors of the robot for various purposes. The use of multiple robots enables redundancy and offers opportunities to increase efficiency.

There are certain reasons for choosing several robot systems:

1. Some missions require a team.
2. You can complete tasks faster.
3. Increases endurance.

4. Multiple bots allow for more versatile and creative solutions.
5. Can handle incomplete communication.

Compared to other distributed systems, multi-robot systems are more complex because they have to deal with a real environment that is more difficult to simulate because it is unpredictable, dynamic, noisy, etc.

The problem of controlling a moving robot and planning its trajectory is dominated by the fact that the robot's knowledge of the world is constantly increasing. This is due to variability in the environment itself, errors in calculating the position of the robot over time, and poor ability to make accurate and precise environment measurements. The problem is further complicated by the complexity of the robot system. The robot must integrate sensor information about itself and its environment from the many categories of the faulty sensor. Its control functions are limited and restricted. It must ensure its security in real-time in case of sudden and unexpected changes in the environment and at the same time perform its tasks at the lowest cost or in the shortest time. In addition, the robot must learn its environment when it encounters it.

As the field of robotics evolves, more and more of these challenges have been addressed simultaneously. In the 1950s, the earliest robots could achieve only, basic movements. In the 1970s and 80s, optimal implementation of global goals dominated active research. Later, real-time constraints gained attention at the expense of optimal design. In recent years, field robotics has moved away from the controlled laboratory and former environment to outdoor environments where less and less prior knowledge of the construction of the world can be assumed. Sensing has moved from SONAR and IR to resolution and model-able LIDAR scanning, colour vision both in and out of stereo range. In addition, the robots accelerated dramatically, from 1969,2 meters per hour to 2005, 70 miles per hour.

The robotics community continues to be challenged by several problems:

1. unknown unstructured environments,
2. sensor errors,

3. control limits,
4. real-time guarantees,
5. global goals,
6. dynamic obstacles,
7. Dynamic work on these questions.

2.2 Notable Contribution

In this paper, we introduce a reactive algorithm to avoid the collision of dynamic, random-shaped obstacles that are suitable for unicycles and non-wheelchair vehicles. Unlike the majority of reactive methods, this method takes the exact shape into account, allowing the vehicle to use spaces that are not occupied by any obstacle. This is preferable to a circle or ellipse approximation, as this can be an excessively conservative operation. We have explicitly provided the conditions in which collision avoidance can be mathematically demonstrated, and we have verified the results with numerical simulations. Because path planning is NP-hard, it can also be solved by multidimensional algorithms. In this article[21], we suggest a multi-dimensional path planning algorithm that consists of 3 steps: 1. Level the road. 2. In Step 2, all optimal, feasible, and non-infeasible points generated by PSO-GEO are integrated into the local search method. 3. The last step (Step 3) relies on collision avoidance detection algorithms. When the mobile robot finds an obstacle in its detection circle, it uses collision avoidance to avoid it. This method is further enhanced by the addition of a mutation operator, which further improves the certainty, length, and smoothness of a mobile robot's path. Various simulations were conducted under different environments to test the proposed algorithm's feasibility. The results showed that the algorithm generates more feasible paths over short distances and has overcome previous art. In this paper, we present an advanced real-time route planning method for heterogeneous multi-robots composed of many UAVs and UGV systems. The 3D environment is modeled as neuron topology, his map is based on the grid method combined with a bio-inspired neural network, and a new 3D Dynamic Mo-

tion model for multi robots is created based on an improved Dragonic algorithm (DA). Robot motion is optimized based on the activity of neurons in a bio-inspired neural network for real-time routing. Some simulations were also performed. The results demonstrate that the proposed method guides heterogeneous UAV / UGV systems to targets efficiently and has better performance in real-time routing tasks compared to conventional methods. Teleoperated or autonomous robots are playing an increasingly important role in civic applications. Among the various domains in which robots can have a more significant impact than human operators, search and rescue activities (SAR) are a prime example. In particular, multi-robots have the potential to improve the effectiveness of SAR personnel by establishing emergency communication networks extending the search for victims Initial assessment and mapping of environment monitoring, and monitoring of SAR operations in real-time. Heterogeneous and cooperative multi-robots offer the greatest benefits because SAR activities span a wide range of environments and contexts. In this paper, we will explore and evaluate current multi-robotic SAR assistance methods from an algorithmic point of view. Collision-free path planning within constraints with minimum processing time and SMOA draws inspiration from the way slime moulds move when encountering prey. Using adaptive weights, its mathematical model simulates optimal paths for capturing prey and food as slime moulds spread toward food with good exploration and exploitation tendencies. I performed a simulation for this purpose. Matlab 2020a SMOA's results were compared with the results of PSO (Precision-Shared Optimisation), FA (Flexibility-Based Optimisation), SFLA (Sub-Linear Functional Analysis), and ABC (Structural-Linear Functional Autonomic Approaches). The modified SMOA was found to be less time-consuming to generate than the other collision-free approaches. Robot path planning has been a key topic in the development of autonomous robots. There have been many advances in the field, and many bio-based algorithms have been developed with great interest. In this survey, we focus on three main approaches to robot path planning: Swarm Intelligence, Evolutional Algorithms, and Neurodynamics, and compare their

advantages and disadvantages. To solve local minima/target unreachability problems of the traditional APF technique for the multiregional cooperative formation as well as global route planning we propose to change the Pulsed field function and add a dynamic virtual destination point. Better state transfer functions based on heuristic data of the APF technique and optimised Pheromone Concentration updating algorithms increase convergence speed and overall optimization accuracy of Ant Colony Optimization (ACO)[26]. To find the best path for the Leader robot from the start point to the end point, we use a hybrid algorithm combining APF and improved ACO. Based on Leader robot path planning, we develop a cooperative multiregional formation control approach that integrates graph theory with a consistency algorithm. For example, AGVs at the warehouse for logistics New method for multi-robot path planning in unknown environment Multi-robotic MOPSO method Concise, safe, and flexible .Given the uncertainty of the environment, a robot should only be able to determine the direction of its movement based on the data collected by its sensors so that the optimum path between the starting position and the target position can be determined at the end of its algorithm [27]. To achieve this goal, an exchange of knowledge among navigating robots is required. In this article, a new concept is introduced, probabilistic windows, which combine current information collected from the robot's sensors and previous robot experience to select paths that the robot believes are more likely to achieve a given goal. In this paper, we focus on a brand new method for generating a foremost collision-loop trajectory route for every robot in a chaotic and unfamiliar workspace using greater particle swarm optimization(IPSO) with Sine and Cosine Algorithms (SCAs). In contemporary work, PSO is associated with a greater awareness of democratic rule in human society and a more grasping approach for determining the foremost role with a subsequent new release of Sine Cosinealgorithms. The proposed method performs well against various complex benchmarks and the results demonstrate that it is more efficient and effective than traditional and state-of-the-art methods. In the proposed set of rules, the main focus is to provide a deadlock loose successive area for each robotic from

its modern area, maintain a good balance between diversification and intensity, and limit the route distance of each robotic. The results of the IPSO-SCA have been compared with the advanced results with the help of IPSO + DE inside the same workspace to verify the performance and reliability of the recommended method. The simulation results and the actual platform results show that IPSO - SCA is advanced up to IPSO DE with inside the shape of producing a first collision loose route, arrival times, and electricity consumption throughout the tour The collaboration and synchronization of several robots is a key issue in robotics research. The poles are thought to be carried by two autonomous robots known as twin robots. The path planning of twin robots is done using a variant PSO. Several parameters are evaluated based on the performance of the twin applied to variant PSO. Run time Number of steps Number of revolutions Distance traveled Deviated path Fitness value of each twin Get closest position along solution path All algorithms are run Pixels plotted to show twin trajectories PSO variant vs. ABCO and differential evolution (DE) PSO variant is superior in distance values Optimization of paths generated by random algorithms Iterative algorithm for raw paths Unlike local post-processing optimizers, this method aims at global optimization of an initial path, where possible, performing drastic topologically update, but rather than re-planing in the entire robot configuration space[30], it limits the search within optimal subset of c. If a better solution is found, it further reduces the subspace. Lazy A* search Efficiently searches the graph representing optimum subspace connectivity Satisfactory and general optimization Computationally attractive This paper also addresses some of the problems related to shortcuts-like algorithms, namely the issue of local shortcuts, and their impact on the result. To evaluate the performance, we compare it with the well-known random shortcuts technique, and we provide extensive experimental results, including different optimization criteria and a wide range of robotic systems, environments, and statistical measures. We have shown a robust post-processing algorithm for random paths. When combined with an effective PRM planner, this algorithm can produce optimized motions while still being

computationally efficient. The goal is to focus the planning on the promising subset, rather than the entire space. Using the lazy A * search algorithm significantly reduces edge validation. This algorithm is computationally equivalent to the efficient RS for all tested examples. However, unlike the RS, it is especially less sensitive to initial solutions. ILR running time is dependent only on one input parameter M. We have shown that this algorithm applies to various robotic systems and cost criteria. We have also shown that the random shortcuts algorithm can lead to a cost that is much higher than what can be achieved. However, it is still interesting under very tight time constraints (e.g. in real-time applications) or when the route is known to be restricted to one corridor. We are experimenting with the neighborhood set for the ILR search. We found that it can significantly improve the running time. However, the size of the neighborhood set is an additional parameter to be specified and is environment dependent. We dropped it for algorithmic simplicity in this work. In this work, we propose a new approach to an online complete coverage solution that is time and energy efficient for mobile robots. Whereas most conventional approaches focus on reducing path overlaps, our approach is to smooth the coverage path to decrease accelerations while increasing the average velocity to achieve faster coverage. Our proposed algorithm takes advantage of a high-resolution grid map representation to minimize directional constraints on path generation. In this case, the free space includes three independent behaviors spiral path tracking and wall following control as well as virtual wall path tracking. As far as the covered region as a virtual wall is concerned, all three behaviors follow the (physical) wall or (virtual) boundary for close coverage. Wall following is performed by a sensor-based reactive path planning control (RPCC) process. Spiral (filling) and virtual wall path is first modeled by their respective parametric curves, and then tracked by dynamic feedback linearization (DRL). For complete coverage, the independent behaviors are linked by a new path-link strategy called a course to fine-constrained inverse distance transform(CFCIDT). CFCIDT reduces computational cost compared to conventional constrained inverse distance transforms (CIDT). CFCIDT applies a

region growing from the robot position to find the nearest unexplored cell and the shortest path to it while limiting the search space. As far as experimental validation is concerned, the proposed algorithm's performance is compared to conventional coverage techniques to prove completeness of coverage; energy and time efficiency; and robustness to environment shape or initial robot pose. In this paper[35], they provide an online Complete Coverage Path planning and control algorithm, based on a high-resolution grid map representation, which allows for a very smooth coverage path by Bezier curve approximations. The smoothness cost function can be minimized by using the Particle Swarm optimization technique, which is based on Nature-inspired optimization because of its real-time capabilities. For space-filling they propose an integrated sequence, which includes spiral path tracking and wall following, as well as virtual wall path tracking with the covered region as a virtual wall. They also propose a path linking method, which is a course to fine-constrained inverse distance transform, to reduce the computational burden of adopting a higher number of grid cells. Cooperative distributed approach for search odor sources in unfamiliar structured environments, with multiple mobile robots. As the robots search and explore the environment, they generate online lo-cal topological maps and share them, creating a global map. This method is based on a decentralized frontier-based algorithm, improved by a cost/utility evaluation function that takes into account the odor concentration at each frontier and the airflow at each boundary. Frontiers with a high likelihood of an odor source are searched and explored in the first instance. The method improves the way robots plan their exploration journey by presenting a prioritization policy. As there is no GLSS and each robot uses its coordinate reference system (CRS) for its localization (37), this paper makes use of topology graph matching techniques to map merge. The proposed method has been tested in simulation as well as in real-world environments with varying numbers of robots and various scenarios. The search time, the exploration time, the complexity of the environment, and several double-visited map nodes were all examined in the tests and the experimental results confirm the workability of this method

in various configurations. The algorithm has been designed to search, explore, and topologically map inside large buildings with parallel and perpendicular corridors (warehouse-like environments) with multi-robot odor sources. The algorithm was tested in a low-scale, real-world test with up to 3 Roomba bots [38]. Below are details on all the additional features needed for such implementation: Motion control Features extraction Classification Localization Map merging Exploration efficiency Odor sensing cues in frontiers selection Navigate to the odor sources Localize them Cooperate by exchanging information in local maps Robot software systems are complex and require a well-structured architecture and programming tools that support it. One common feature of robot architectures is that the systems are modularized into simpler and mostly independent components. These components implement old actions and report events about their state. The proposed robot programming framework includes a tool, Robo Graph, to program/coordinate the activity/tasks of these middleware modules. The project developers use this task programming IDE on two levels: To program tasks that need to be executed independently by one robot The tasks are described using the SIPN editor and stored in the XML file. The dispatcher loads the files and runs the various Petri nets as required. A monitor that displays the status of all running nets is very helpful for debugging and tracing. The system has been used for several applications: a tour-guide robot (Guide-bot), a multi-robotic surveillance (Watch-Bot) project, and a hospital food (Laundry) transportation system using mobile robots. Summary: A new framework for creating mobile robot applications. Developing a new application requires three levels of programming. Basic primitives can be pro-vided into modules which can be developed using various programming languages depending on their nature. Tasks can be defined as SIPN using the RoboGraph IDE for quick definition and debugging. Resources can be shared, coordinated, and planned by the task manager module. A group of robots can move as a single entity performing the following tasks: Obstacle avoidance at the group level Speed control Inter-robot distance modification The flocking controller divides the robots [43] so that they can move in a common direc-

tion and keep the desired distance between robots. The framework is composed of different modules that change the behavior of the groups so that different functions can be performed. The consensus algorithms are based on which robots have more relevant information so that they can agree on different parameters. New modules can easily be designed and integrated into the framework to improve its capabilities. It is easy to implement on any mobile robot that can measure the relative position of neighboring robots and communicate with them. It has been tested with 8 real robots and has been used in simulation with 40 robots to experimentally demonstrate its scalability with increasing numbers of robots. In this article, we have proposed, analysed, and tested a collective movement framework for mobile robots. The framework allows robots to move like a unique entity, agrees on group velocity, avoids obstacles by changing the group direction of movement, and changes the inter-robotic distance. The framework consists of different modules which change the movement-functions. Based on the distributed consensus mechanism, robots can agree on various parameters and also assign their value to the rest when the rest has relevant information[46]. Consensus algorithms for agreeing on the group speed and direction, together with the distributed orientation agreement mechanism, avoid the use of an external objective towards which to move robots and relieve robots from estimating the velocity or heading of neighboring robots. Estimating neighboring robots' velocity or heading is used in the majority of consensus-based movement algorithms. Estimating velocity on actual robots is challenging.[47] In the present paper, we use a new multiregular exploration algorithm, which applies a globally optimized K-means clustering strategy to ensure a balanced and continuous exploration of large workspaces. This algorithm optimizes the online assignment of the robots to targets and keeps them working in distinct areas, efficiently reducing the variance of the average waiting time within those areas. This algorithm ensures that different areas are explored at similar speeds, thus avoiding areas that are searched and rescued much later, which is desirable for many exploration applications such as search and rescue. This paper presents a new KME algorithm for multi-robotic

exploration. The algorithm results in the lowest WTV and the lowest EPV variance. This is demonstrated by comparing the proposed algorithm to different state-of-the-art approaches. Compared to the state-of-the-art approach tested in this paper [49], the proposed algorithm results in the least variance of the average waiting time for regions. This is desirable for a variety of exploration applications, including search and rescue (SAR). Robotic exploration missions require robots to learn policies that enable them to: identify high-level destinations navigate (achieve those destinations) change their behavior (adapt to changing environmental conditions) be able to signal noise/unexpected situations scale to more complicated environments respond to physical limitations of robots (such as limited battery power/computational power) This paper [51] evaluates reactively and learns navigation algorithms for exploring robots that must navigate around obstacles and reach a specific destination in a short amount of time and with a limited number of observations. The results show that with well-designed evaluation functions, neuro-evolved algorithms with well-designed learning functions can achieve up to 50 percent better performance than a reactive algorithm in complex domains in which the robot's goal is to select paths that lead to specific destinations and avoid obstacles, especially when faced with high signal noise from sensors and actuators. This paper shows that adaptive behavior is still effective in domains with very limited resources it can perform better than a set probability distribution based on the failure of sensors and actuators, as the neuro-evolved algorithm learns slowly but converges to higher performance because it can learn on an objective that more directly indicates system-level needs. The path planning problem for mobile robots is considered one of the fundamental problems in robotics, in the case of several robotic systems. The complexity of these systems increases proportionally to the number of robots because all robots have to act as a single unit to complete a single composite task, for example, maintaining a specific formation. The proposed path planner uses a combination of CA and ACO techniques to generate collision-free paths for each robot of a team, while their formation remains absolute or unaltered. The method reacts to obstacle distribution

changes and can therefore be applied in dynamic or non-dynamic environments, without prior knowledge of the environment. The team is split into subgroups, and all the desired paths are generated with the combination of the CA path planner and ACO algorithm. In the event of a pheromone-poor environment, paths are generated using the CA path planner. In comparison to other methods, this method can generate accurate collision-free pathways in real-time, with low complexity, and the implemented system is fully autonomous. The proposed method was tested in a simulation environment using real-world simulation, called Webots. The CA algorithm and the ACO algorithm were used in a team of several simulated robots without central control. The simulation and the experimental results show that precise collision-free paths can be generated with low complexity. In this paper[54], we presented a new approach to solving the problem of path planning in a collaborative robot team. The approach is based on a combination of two AI techniques, namely the CA algorithm and the ACO algorithm. To test the efficiency of this approach, we created a 2-dimensional environment. In this paper, we discuss the positioning, motion coordinate, and test order procedures of new PCB testing equipment. The equipment architecture consists of 4 mobile probes whose movement must be coordinated to avoid collisions with obstacles and each other. The algorithm used to find the optimal path between 2 points in the space with obstacles is based on conventional methods but includes novel ideas to minimize the calculation time. A new form of formation coordination that finds the optimal sequence of successive or simultaneous probe movements and is based on decomposing the sequence of movement into stages where only movements without interference are allowed[56]. The order of the tests can also be considered a traveling salesman problem. The results of various methods are presented when applied to the particular characteristics of the equipment. This paper presents the full problem of path planning and motion coordination as well as the method to get the optimal tour. The tasks were programmed to include a flying probe system which usually handles several thousand tests involving 2, 3, or 4 probes [57]. Path planning g algorithm is based on known

techniques but with different modifications. The goal function to be optimized contains terms that lead to paths that give better optima during motion coordination problems. The procedure to reduce vertices is included to reduce the time needed to traverse an obstacle. A Practical Viable Approach for Conflict-Free, Coordinated Motion Planning of Multiple Robots The proposed method is a two-stage decoupled approach that can provide the required coordination among the robots participating in offline mode. In the first stage, the collision-free path concerning stationary obstacles is achieved by an A* algorithm for each robot. In the second stage, the coordination among several robots is achieved by the resolution of conflicts based on the path modification approach. The paths of the conflicting robots are modified according to their position in the dynamically calculated path modification sequence. To evaluate the effectiveness of the methodology, different strategies are used to achieve coordination among robots. These strategies include fixed priority sequence allocation for each robot's motion, reducing the speed of the robot's joints, and introducing a delay in starting each robot. The performance is evaluated based on the length of the path each robot traversed, the time it takes the robot to complete the task, and the computational time. Two case studies were conducted that looked at the tasks with three robots and four robots. Based on the results obtained from a real-world simulation of a multilevel robot environment, the proposed approach guarantees fast, concurrent, and non-conflict coordinated path planning for multilevel robots. The offline decoupled method for coordinated manipulation of several robots calculates the dynamic priorities to change the path and solve the conflicts. It is an easy and effective method to achieve a conflict-free, coordinated path for manipulating several industrial robots working in a joint workspace. The multi-motion planning method used an incremental A*. algorithm to search for discrete configuration space-time using intelligent CST to store the robot's current position in its CST as well as the path followed by each robot to effectively transfer information to the motion planner. The dynamic priority computation for changing the path ensures the concurrent movement of all robots regardless of the number or

type of obstacles present in the environment. In this paper, we solve the sweep coverage problem by deploying autonomous mobile robots using a decentralized control algorithm. Sweep coverage is accomplished by coordinating robots to follow a path that is a priori unknown to vehicles. Motion coordination algorithms are developed based on simple consensus algorithms and their effectiveness is demonstrated by numerical simulations. The proposed algorithm has military and civilian applications. Sweep coverage could be applied in several applications, such as Reconnaissance Minesweeping Maintaining maintenance inspection Cleaning ship hulls Multi agent minesweeping In this paper[61], they aim to solve the sweep coverage problem by deploying an autonomous robot network that could be similar to a minesweeping operation. The cost of operation should be reduced by reducing the communication and detection capacity of the robots. Therefore, autonomous robots should operate in a distributed mode, unsupervised. Control of these autonomous robots is a decentralized control problem because each robot only has local information for control. In this paper, they developed a set of decentralized control laws for coordinating a mobile robot sensor network. The control laws are based on consensus algorithms that are relatively easy to compute and easy to implement. To achieve sweep coverage, according to the control laws, the group of sensor-equipped mobile robots will form a sensor barrier moving perpendicularly toward the path at a specific speed. The separation between the pair of robots within the sensor barrier can also be adjusted. The control algorithm can be illustrated by numerical simulations for several scenarios. In this paper [63], we propose an important concept 'local shortest path' for mobile robots' and demonstrate that a compact V-graph (V-graph) can be constructed with a size of O(M2 + N) where M is the number of convex components, and N is the number of polygonal vertices of obstacles. In addition, we propose an R-graph (R graph) with a size O(N*M2) registering reach capacity between vertices on the local shortest paths. The R-graph is independent of obstacles in the environment and does not depend on the size of the mobile robots. Therefore, even if the robot's size or the required safe distance between robots and obstacles changes,

it is still possible to efficiently plan a robot's path by picking up the robot's reachable heights in R-graphs without having to reconstruct R-graphs. Finally, several simulations verify the usefulness of this algorithm. This paper proposed a significant definition of the 'local shortest path' for mobile robots and demonstrated that a smaller V-graph (O(M2+N)) can be constructed based on this definition. To resolve the problem of 'v-graph' that depends on robot size, we also proposed a 'reachability graph' with size O = N*M2', which indicates the reachability of the robots between the vertices on the local shortest paths, as the proposed reachability graph only depends on obstacles in the environment, but not on robot size. There is no need to reconstruct the reachability graph if the safe distance between robots has changed or if the robot's size has changed. In addition, due to the sequence of arcs of circles and the common tangent segment of the arcs in the path, motion can easily be realized in the robot control. Finally, the path planned is always the shortest locally, and the shortest path can be found efficiently by some graph searching approach such as the algorithm A*. In this paper, we present a new multidimensional approach to path planning. Multi-resolution path representation does not require explicit configuration space calculation and includes an evolutionary algorithm to solve a multimodal optimisation problem. Multiple alternative paths can be generated at the same time. The multi-resolution path representation lowers the expected search time for the path planning problem and thus reduces the total computational complexity. The evaluation function introduces resolution-independent constraints based on obstacle proximity and path length. The resulting path planning system has been tested on problems of 2-D, 3-D, 4-D, and 6-D degrees of freedom. The multipath algorithm has been demonstrated on many 2-dimensional path planning problems. The simulation results demonstrate that this multi-dimensional path planning algorithm has good performance. In this paper, we propose an Instant Goal approach for collision-free obstacle-following obstacles of arbitrary shape in unknown environments. We also provide a vector representation for efficient knowledge representation and manipulation. This not only saves a lot of memory but also matches the physical properties of

the range sensors. We also introduce the concept of "instant goals", where the robot can perform boundary following "just like a human", with additional measures to make sure that the robot is "moving" along the boundaries, even if the obstacles are of arbitrary shapes and disturbing obstacles present. At the same time, collision checking is performed and, when necessary, collision avoidance is integrated. Based on the boundary-following approach, we design a realistic sensor-based path planner with a global convergence property for the robot that can acquire discrete and noise range data. We also provide realistic simulation experiments to prove the effectiveness of our proposed approaches. Several questions have been addressed in this paper: 1. A vector representation for the local environment 2. Saves a lot of memory 3. Is suitable for behavior generation 4. Is suitable for obstacle avoidance The Instant Goal methodology is designed to keep the robot moving along an obstacle of any shape even when other disturbing obstacles are present. Based on this boundary-following approach, a realistically convergent path planner is proposed for robot navigation in unstructured and complex environments. In this paper, [68] describes in detail the implementation of a Kalman-based active observer controller (AOB) for path following of WMRs subject to non-holonomic constraints. Some particularities of this control strategy are: It is used in discrete mode It is robust to uncertainties and disturbances (e.g. due to input-output feedback - linearization method used in discrete mode) It has been developed for continuous mode The performance of this control algorithm is verified by computer simulation and compared to other control strategies (e.g., pole placement controller [PPC], PPC with Kalman filter observer [CKF] The AOB design showed better results in path following compared to the classical control strategies (CKF and PPC) even when the system was subjected to disturbances. This is the first time the AOB design was used for the path following a WMR. The results demonstrated the validity and efficacy of the control design even when the robot's physical parameters (e.g. mass) were changed. This paper addresses the modeling and control problem of a robot convoy [69]. The guidance laws techniques are used for mathematical formulation. The guidance laws

are velocity pursuit, deviated pursuit, and proportional navigation. Velocity pursuit equations model a robot's path under various sensor-based control laws. The tracking problem is systematically studied based on this technique. These guidance laws are used for decentralized control laws for angular and linear velocity. The control law for the angular velocity is derived directly from the guidance laws, after taking into account relative kinematic equations between consecutive robots. The other control law keeps the distance between consecutive robots constant by controlling linear velocity. This control law can be derived by taking into account the kinematic equations between consecutive robots under the given guidance law. We discuss and prove the properties of this method. Simulation results prove the validity of this method. We use 3 different methods and derive 2 control laws per the following robot. The control law for orientation angle is derived directly from guidance laws equations, after the kinematic equation between consecutive robots. 2 control law for linear velocity is designed to keep the distance between consecutive robots constant. In this case, kinematic equations are used between two consecutive robots subject to guidance law. It is demonstrated that for all 3 strategies, each subsequent robot follows its lead robot's angular velocity, where each subsequent robot tracks its linear velocity, and its lead robot tracks linear velocity. Simulation proves the validity of this approach. How to Generate Real-Time Trajectories for Platoons of Autonomous Vehicles How to generate real-time trajectories for redundant robot manipulators The partially decentralized nature of this approach allows each vehicle to compute its trajectory independently in real-time using only locally acquired information and low bandwidth feedback from a system external to the platoons. Their work was motivated by applications where communications bandwidth is extremely limited, for example, autonomous underwater vehicle (AOV) platoons. Our trajectory generation approach does not depend on the number of vehicles in a platoon. This means that platoons made up of large numbers of vehicles can be coordinated even when communication bandwidth is very limited. In this paper (70), we present a decentralized treatment for a redundancy-based cooperative pla-

toon control algorithm. The results show that the redundancy-based approach can be decentralized, and the secondary objectives (local autonomy, obstacle avoidance) can be achieved through inter-vehicle communication as needed. The main goal of this approach was to reduce communication demands. This was accomplished by regulating the platoon as a whole rather than the individual vehicles. An exogenous system is expected to measure the performance of the platoons and send signals to the platoons that each vehicle uses to control a local path generator algorithm. The total amount of communication required is relative to the number of functions to be controlled and does not depend on the number of vehicles used in the platoons. Therefore, this approach is well-suited for large-scale platoons. In this paper (Rami Al-jarrah et al., 2015), the research focused on the path planning of multiretroviral and collision-free coordination using probabilistic neuro-fuzzy. This approach uses subtractive clustering and least square estimation to create fuzzy rules for understanding the relationship between input data and output data. The leader and follower approach uses a neural network to fine-tune membership function attributes and an AN-FIS to fine-tune fuzzy rules. The research was carried out in a dynamic environment, where obstacles will be continuously moving. Sensors are used by the leader and followers to understand the environment. Simulation results demonstrate the feasibility of the proposed work and increase the throughput of a navigation system. In this paper, (Neil Mathew et al. (2015))discuss package delivery in urban areas with multi-robots using a Travelling salesman problem with two variations, enumeration, and reduction, using several static warehouses. Two vehicles were used for shipment and delivery (carrier truck and carried MAV). The main objective of this paper is to find the optimal path for a MAV/QUADROTOR to deliver products at all locations. The goal of this research was to address the limitations of the heterogeneous delivery problem and to formulate it as an optimised path planning concern. Using this approach, we can see a 50To enhance the convergence speed and smoothing of the initial path of the proposed method, they have implemented a multivariate heuristic function. The results of the experiment are

validated in MATLAB and ROS. The main focus of this paper is on route planning in a dynamic environment with variable barriers. The overall goal of this article is to reduce the overall cost. This strategy is simple and efficient in both a dynamic and a static context. The proposed approach can overcome any kind of obstacle, regardless of size and shape (although in this paper we focus on the obstacles to have a circular form). In this paper, Jingjin Yu et al. (2019) present their work on path planning in a well-connected environment with multi-robots. They have created a polynomial-time algorithm that is split and grouped to solve problems on grids and generates, on average, constant factor scheduling globally optimal for all occurrences in the challenge. SaG approximates results using a sublinear calculation time and is an optimal case O (1)- approximation procedure. From a technical point of view, combining global dissociation with network flow strategies may be more effective in scenarios with a lower robot population, bearing in mind that only the situation with the highest robot frequency was considered. In this article,(Rahul Kala (2011)) is primarily proposing an evolutionary genetic programming approach to address this issue. First, they proposed a combined centralized and decentralized programming model. The proposed model uses collaborative progression concepts to create pathways that are optimal in timing. Second, they described the paradigm and challenges of multi-speed map-based motion planning for more than one robot. Third, they proposed the use of external memory for the optimization of regional paths. External memory is generated, updated, and accessed to identify whether robot pathways are interrupted. Fourth, they proposed to use the evolutionary approach's wait for robot feature. The main focus of the article is to model the journey of the mobile robot using the grid map. A mobile robot with SLAM with functional reactive movement (PRM) is a presumption for path planning. These problems are not addressed in the paper. The author improves upon the A-star technique. In the main section of the paper, numerous adjustments are introduced. For example, Basic Theta*(Phi*) is introduced in the main section. Changes are mainly related to computational efficiency and the maximization of path optimization. Specific

adjustments are tested in various scenarios with varying degrees of environmental complexity. This assessment allows you to choose the optimal path-planning strategy for each case. Second, they outline the paradigm and challenges associated with multispatial, map-based motion planning for several robots. They use the evolutionary approach's wait-for-robot feature. Third, they propose external memory for the optimization of regional paths. External memory is generated, updated, and accessed to see if robot pathways are interrupted. In this paper, (Kaushlendra Sharma (2021)) examined the issue of multi-robot path planning with cooperation. This is a serious issue on its own. Trying to solve these two problems at the same time is a challenge. By ignoring the problem of path length, just coordination can be achieved in a much easier way. On the other hand, if there are no restrictions on robots colliding with each other or sharing their segment of the route with several other robots, it can be possible to optimize the path length of each robot. However, in a multi-robot setup, these two problems are contiguous and need to be solved at the same time and also for the successful execution of any operation. Therefore, it is necessary to carefully manage each parameter while maximizing others. We have used the warehouse system in this essay. The proposed method of smart choosing has been used to track and validate the results. This study compares different path-finding techniques (A* algorithm and Modified A* algorithm). The percentage increase in path length and the percentage decrease in the calculation are observed and measured in this study. The updated A* algorithm reduces processing time by a minimum of 65 % with a maximum path length increase of 3.4 %. When the difference between the origin and first barrier is more than the righteous line, and inspection of each consecutive value is included in the heuristic approaches, modified A* performs better than the original A* algorithm. This study is based on the publication "Time Efficient A* algorithm"(by Akshay Kumar Guruji et al., 2016.) (Akshay Kumar Guruji et al.,2016)This study compares different path-finding techniques and the publication suggests time efficient A* algorithm. The percentage increase in path length and the percentage decrease in the calculation are observed

and measured for both algorithms, A* and Modified A*. In this study, the updated version of the A* algorithm reduces the processing time by a minimum of 65, with the maximum path length increase being 3.4 % When the difference between the origin and first barrier is more than the righteous line, and inspection of every subsequent value in heuristic approaches is included, modified A* performs better than the original A* algorithm. (xin Chen guruji et al., 2019) In this paper, we discuss the problem of multi-robot coordinated path planning in an intelligent warehouse, we discuss the question of multi-robot path planning, we outline the basic elements of an intelligent warehouse and we provide a provide graphical representation of the whole system. In the real warehouse scenario, consistent robot driving is required, so it is proposed to adjust the architecture of the allocation column in the WHCA* method. The persistent time for each location is reserved and this information is stored. In most cases, continuously changing the planned course every time the algorithm is called will allow the robot to move at a higher speed, based on our experiments to see if we can reuse the robot's path knowledge every time the algorithm calls. We can reduce the number of turns on the path as well as the transit times while ensuring that the robot does not travel any further by using a turning factor as a metric. We enhance the functioning of the smart warehouse systems using these techniques. (Michal Čáp et al.,2015), In this paper, we propose a method for mult-robot trajectory planning i.e., prioritized planning. In this paper, we have looked at the effectiveness of 6 alternative algorithms that employ the idea of ordered planning and discussed their attributes in detail. PP, SD-PP, and RPP are already existing algorithms that we have discussed in the past, but the other 4 methods are our first contributions. This paper's first contribution is a modified version of RPP and a structured demonstration that the algorithm is guaranteed to solve if each robot has a route that gets it to its destination, avoids its start positions from robots with low priority, and avoids its goal positions from robots with high priority. We have shown that this requirement is satisfied when a robot is moving between two endpoints of a well-formed infrastructure. This is important because human made environments

are usually constructed as well-formulated infrastructures and it is possible to determine coordinated paths for the robots working in such environments using the RPP algorithm. The second implication of this paper is a unique asynchronously decentralized realization of the classic prioritized official plan and a formal demonstration of the algorithm's termination guarantee. In addition, we showed that the asynchronous, decentralized implementation of better-prioritized planning can reliably design coordinated trajectories within well-designed infrastructure and can do so reliably under similar conditions as its centralized counterpart. (Cardarelli et al.,2017) This study designed a cloud automation system that collects data from multiple sensory sources. Data about all objects in the industrial environment is then defined into a global live view of the environment. This was then used to enhance the local sensing capabilities of AGVs, increasing the efficiency and flexibility of the coordination of the AGV motion. The data from multiple sensors is combined to create the global live perspective. The global live perspective combines sensory data after processing locally and provides data at the geometrical and decision levels. More specifically, multiple sensors in a real industrial environment were used to verify the suggested architecture. The global live view allowed for the safe operation of the AGV. While the suggested architecture is only applied to warehouse settings, the concept can be applied to broader intelligent manufacturing applications. Future work will attempt to include data about the status of manufacturing lines and machines within the global live view. (Akhlaqur Rahman et al.,2019) This work aims to develop a unique multi-layered task offloading decision-making technique that takes into account four factors: choice of the task to offload (i) choice of the robot to offload (ii)choice of place to offload/perform the task (iii)choice of the access point to offload (iv) To support our task offloading scheme, we provide a paradigm for internet-connected multi-robots networks where the process of offloading can be enhanced by the lead robot with the help of other nearby robots. The study focused on a 36-cell workspace warehousing scenario, with a 40-node job flow that is based on a 'parcel selection and circulating' application that the main robot will carry

out. A multi-layered evolutionary algorithm approach is designed to deal with the offloading decision of each job as a component of a common optimization problem that takes into account robot motion, network access, and local collaboration. The scheme's output is compared to two verified benchmarks to evaluate its performance. The results demonstrate a significant increase in overall system performance due to movement, connectivity, bandwidth estimation, and robot-robotic interaction (locational offloading), resulting in energy-efficient cloud-based offloading, faster task execution, and better resource utilization.

The citation for the base publication taken into consideration for the study will be found in the first section of the Table of Entries. Single robot/multi-robot is the name of the next column. Another division of this idea occurs when one or more robots are used to carry out a specific task or to meet various objectives in a simulation or environment. The next column represents the area where the proposed approach is implemented and illustrated. The algorithm that is employed in the proposed approach is noted down in the immediate column The very next column presents information about the environment. And the last two columns discuss the advantages and drawbacks of the proposed work..

The bar graph in Figure 2.1 shown above is based on the research study stated in Table 1, the graph's x-axis shows the year, and it was drawn using blocks of 10 years, each of which contained at least 4 of the most frequently referenced papers. The total number of citations that papers have obtained throughout the specified year reflects the count at the apex of each pillar.

Reference	No. of Robots	Area Environment	Technique Used	Type of Environment	Pros	Cons
František DuchoĖ et al.(2014)	Single robot	Grid Map	Modified A* algorithm	Known and Static	Simple and relatively fast and can be modifiable easily	Only applicable to quickly path finding scenarios
Kaushlendra Sharma et al.(2021)	16	Warehouse	GA and ILP	Unknown and Static	Improved performance, average path length and elapsed time	Can't work well with large warehouses
Akshay Kumar Guruji et al.(2016)	Single Robot	Fast processing applications	Time efficient A* algorithm	Known and Static	Reduction in execution time and improvement in path length	Only applicable to applications where little cost of path length is present
Toolika Arora et al.(2014)	Single robot	Rectangular Map	Genetic Algorithm	Unknown and Dynamic	Achieved good execution time and path length	Genetic Algorithm is not applied on the complete space
Xin Chen et al.(2019)	Multi Robot	Warehouse	Windowed Hierarchical Cooperative A* Approach	Unknown and Dynamic	Only fewer turns are needed and reduced travel time of robot	Low efficiency
Michal Čáp et al. (2015)	Multi Robot	Empty Hall, Office Corridor and Warehouse	Revised Version of Prioritized Planning and ORCA	Known and Dynamic	Message Exchange between Robots is reliable and instant	low success rate when high number of robots are used

Part-2

Chapter 3

Nature-Inspired Approaches for Path Planning and Their Analysis

For several years, various researchers and scientists have provided several methods for navigational approaches. Various methods used for navigation of a mobile robot are generally classified into two categories i.e. classical and reactive approaches. Classical approaches

Initially, classical approaches were very popular for solving robot navigational problems because, in those days, artificial intelligence techniques had not been developed. By using classical approaches for performing a task, it is observed that either a result would be obtained, or it would be confirmed that a result does not exist. The major drawback of this approach is high computational cost and failure to respond to the uncertainty present in the environment; therefore, it is less preferred for real-time implementation.

Different algorithms were studied and understood to work with path planning problems, and only some of the algorithms were explained in detail in this section as these approaches are adopted in most of the research articles [28]. Other than these algorithms, many other approaches were there to solve the path planning problem which including Bee colony, Firefly, Bacteria Foraging, Cuckoo Search, BAT, Grey wolf, PRM, D*, Dijsktra etc [29]., have also been employed in a few research studies.

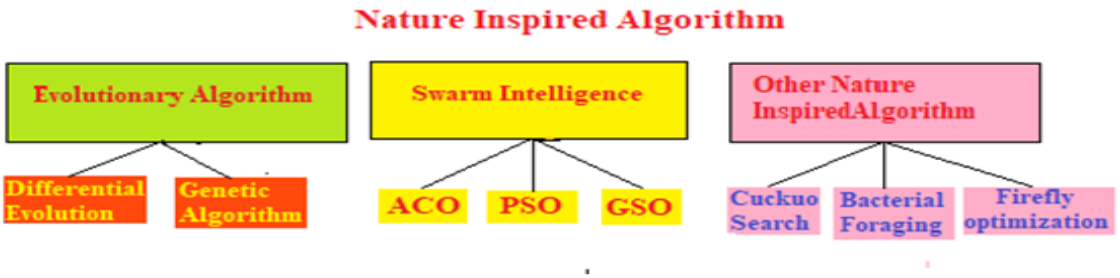

Figure 3.1: Mobile Robot Navigation Approaches inspired from nature

In this section, we mainly focused on the notable contributions of most of the authors in this path planning domain. We explained each algorithm with its pseudo code, flowchart and process flow in order to make each approach easy to understand.

3.1 Particle Swarm Optimisation (PSO)

PSO is a derivative-free technique, just like other heuristic optimisation techniques. PSO is easy in its concept and coding implementation compared to other heuristic optimisation techniques. PSO is less sensitive to the nature of the objective function compared to the conventional mathematical approaches and other heuristic methods. PSO has a limited number of parameters, including only the inertia weight factor and two acceleration coefficients in comparison with other competing heuristic optimisation methods. Also, the impact of parameters on the solutions is considered to be less sensitive compared to other heuristic algorithms. PSO seems to be somewhat less dependent on a set of initial points compared to other evolutionary methods, implying that the convergence algorithm is robust. PSO techniques can generate high-quality solutions in shorter calculation times and with stable conver-

gence characteristics than other stochastic methods.

The major drawback of PSO, like in other heuristic optimization techniques, is that it lacks a solid mathematical foundation for analysis to be overcome in the future development of relevant theories. Also, it can have some limitations for real-time ED and other applications, such as 5-minute dispatch considering network constraints since the PSO is also a variant of stochastic optimisation techniques requiring relatively longer computation time than mathematical approaches. However, it is believed that the PSO-based approach can be applied in offline real-world ED problems such as day-ahead electricity markets. Also, the PSO-based approach is believed that it has a less negative impact on the solutions than other heuristic-based approaches. However, it still has the problems of dependency on the initial point and parameters, difficulty in finding their optimal design parameters, and the stochastic characteristic of the final outputs. Particle swarm optimisation (PSO) is a popular optimisation technique inspired by the collective behaviour of social animals such as birds flocking or fish schooling. It can be used for a variety of optimisation problems, including path planning.

In path planning, the goal is to find the optimal path for a robot or agent to move from a start position to a goal position while avoiding obstacles in the environment. The PSO algorithm can be used to search for the optimal path by treating each possible path as a particle in the swarm.

Each particle represents a candidate solution to the path planning problem, with its position representing the path and its velocity representing the direction and magnitude of the movement of the particle. The algorithm evaluates the fitness of each particle, which corresponds to how well the path avoids obstacles and reaches the goal. The fitness function can be designed to consider various factors such as the length of the path, the clearance from obstacles, and the smoothness of the path.

The particles move through the search space, adjusting their position and velocity based on their own previous best position and the best position found by the swarm so far. This creates a swarm behaviour that can efficiently search the solution space

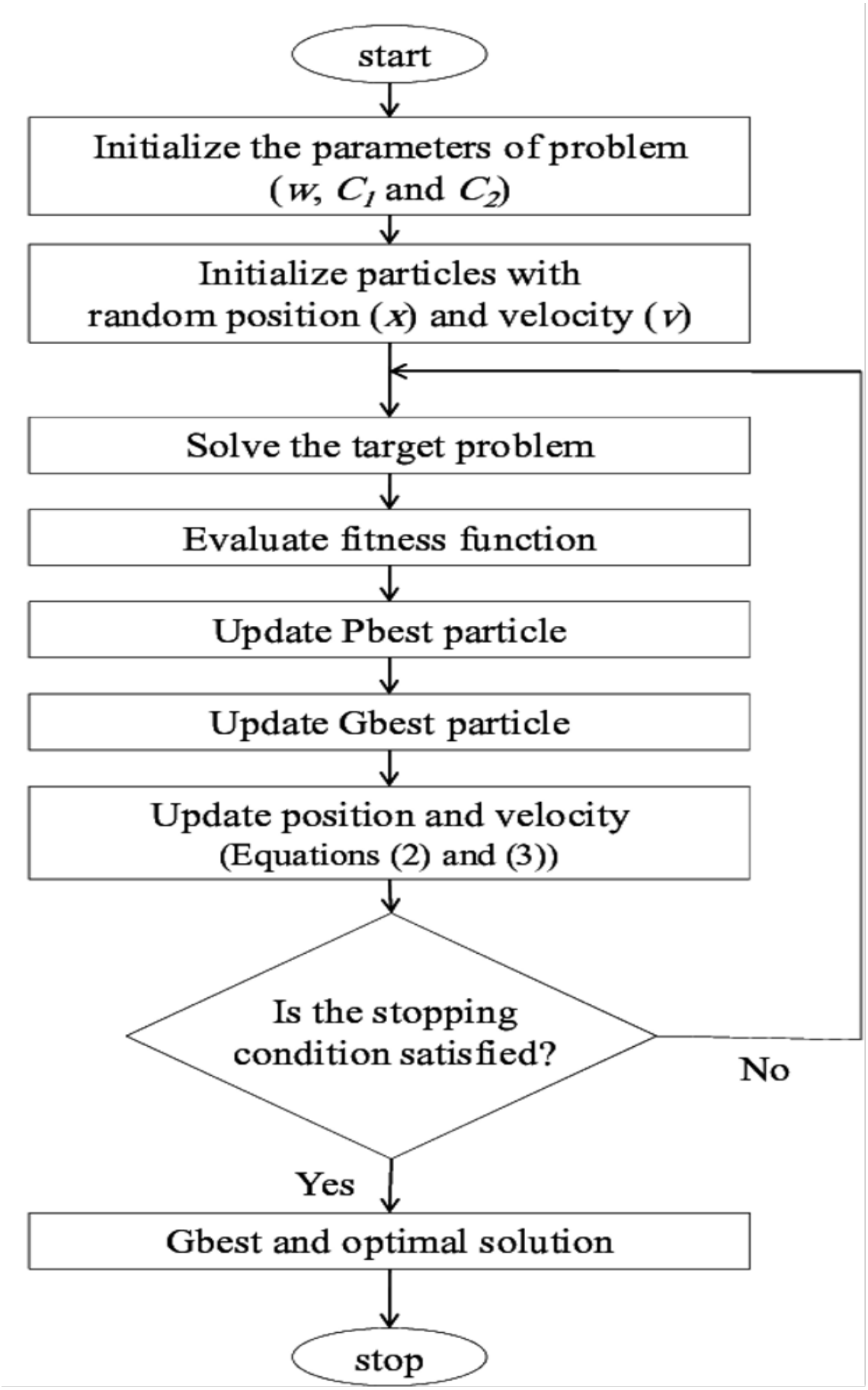

Figure 3.2: Process flow in Particle swarm Optimisation

Table 3.1: Advantages and Disadvantages of Particle Swarm Optimisation

S.No	**Advantages**	**Disadvantages**
1	Inherent parallelism	Theoretical analysis is difficult
2	There is no central control in the colony	Sequences of random decisions (not independent)
3	Positive Feedback accounts for rapid discovery of good solutions	Probability distribution changes by iteration
4	Efficient for Travelling Salesman Problem and similar problems.	Time to convergence uncertain (but Convergence is guaranteed.
5	can be used in dynamic applications (adapts to changes such as new distances, etc)	Research is experimental rather than Theoretical

for an optimal path.

Particle swarm optimisation (PSO) is direct in its pursuit of the optimal answer to the issue at hand. It varies from other optimisation strategies in that it only needs the fitness value and does not depend on the direction or any dependent version of the goal [31]. The space of a particular issue or function is filled with a variety of small, simple things called particles, each of which assesses the fitness function where it is currently located. Each particle then calculates its travel through the solution space by fusing a historical component of its present and best (best-fit) coordinates with the coordinates of one or more swarm members, along with some random disturbances [32]. Once every particle has been shifted, the process moves on to the next iteration. The swarm will eventually move near to the fitness function's maximum, much like a group of birds looking for food together.

Algorithm 3.1 Robot Path Planning using Particle Swarm Optimisation

Require: $P \leftarrow List\ of\ points\ L : (l_1, l_2, \ldots\ldots l_n)$
Ensure: $d \leftarrow L_n - L_{n-1}$
Procedure minPath :
for $L \in P$ **do**
if $L \neq any\ obstacles\ or\ danger\ zone$ **then**
$X = collision\ free\ path.add(p)$
end if
end for
for $L \in$X **do**
$Calculate\ D_t(L_n - L_{n-1})$
if D_t ¡ min(d) **then**
$min(d) \leftarrow D_t$; $d \leftarrow$L
end if
Return d
End Procedure

The optimal route discovered by the swarm can be utilized for robot or agent navigation when the optimization process is finished. Because the PSO method can qu adjust to changes in the environment and generate new optimal paths as needed, it is very helpful for path planning in dynamic and uncertain situations.

3.2 Ant Colony Optimisation (ACO)

Another optimization method that can be applied to robot path planning is ant colony optimization (ACO). It draws inspiration from the way ants communicate with one another and locate the quickest route between their nest and food sources using pheromone trails. The environment is represented as a graph in ACO-based robot path planning, with nodes standing for the robot's location and edges for potential routes connecting the nodes. Finding the shortest route between the start and target nodes while dodging environmental barriers is the aim.

The ACO algorithm works by simulating the pheromone trail left by ants on the graph. Initially, the pheromone level on all edges is set to a small value. A group of virtual ants is then released from the start node, and each ant chooses its next move based on the pheromone level on the available edges. The probability of an ant selecting a particular edge is based on the pheromone level and the dis-

tance between the current node and the destination node. Ants are biased to select shorter edges with higher pheromone levels. As ants move through the graph, they deposit pheromone on the edges they travel. The amount of pheromone deposited is proportional to the quality of the path taken by the ant.

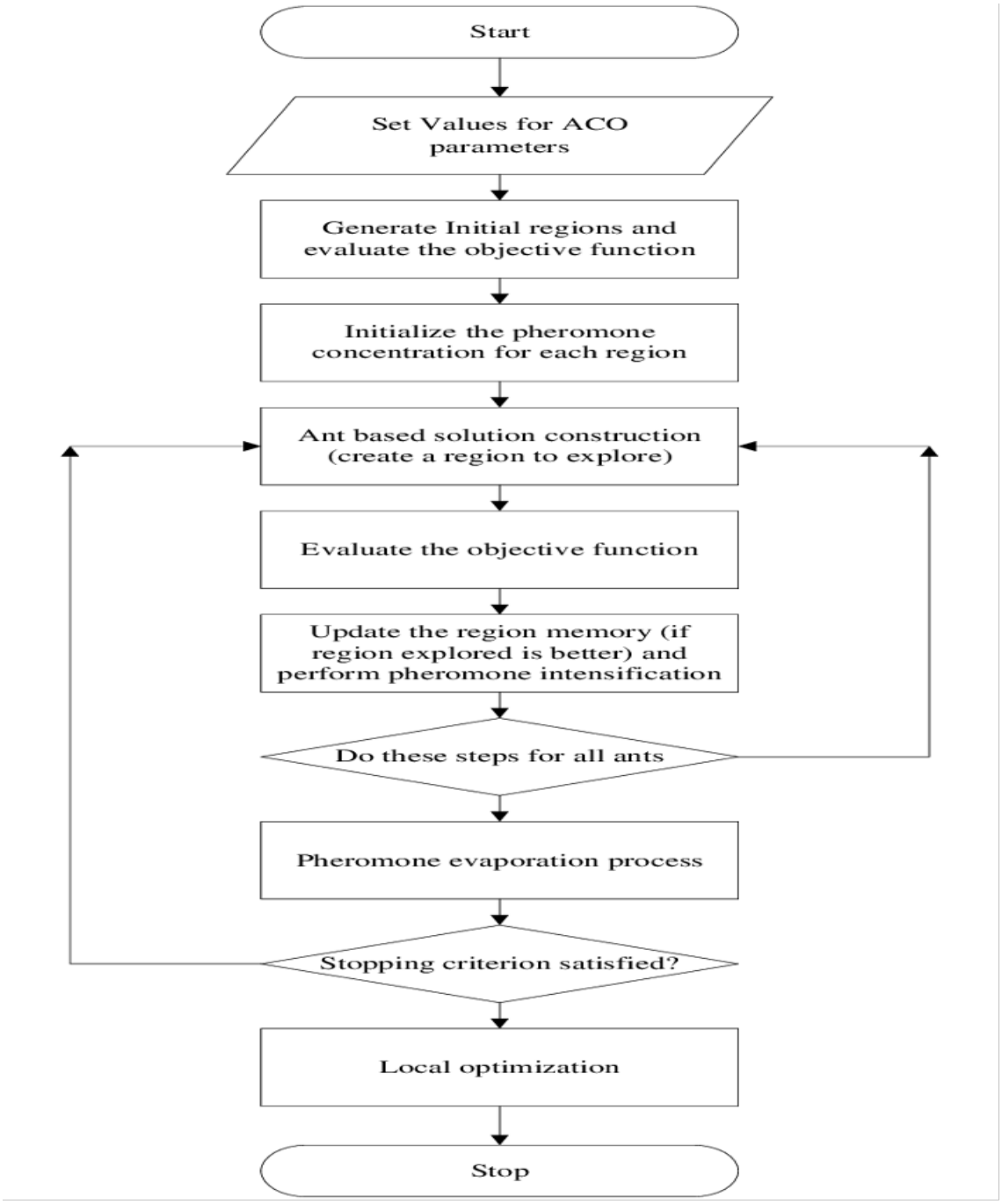

Figure 3.3: Working Flowgraph of ACO

Over time, the pheromone level on the edges of the optimal path increases, while the pheromone level on other edges decreases. This creates a positive feed-

back loop, where the ants are more likely to choose the optimal path due to the higher pheromone level, and the optimal path becomes even more reinforced by the pheromone trail. Once the algorithm converges to a solution, the path with the highest pheromone level is selected as the optimal path from the start node to the goal node. This path is then used for robot navigation.

ACO-based path planning is effective in finding optimal paths in complex environments with multiple obstacles. It is also capable of dynamically adapting to changes in the environment by recalculating the optimal path as the pheromone trail changes.

Ant colony optimisation is a probabilistic method for determining the best paths to take, It is a population based metaheuristic algorithm that can be used to find approximate solutions to difficult optimisation problems. The ant colony optimisation approach is used in computer science and research to solve a variety of computational challenges. Marco Dorigo introduced ant colony optimisation (ACO) in his Ph.D. thesis in the 1990s. This method is based on an ant's foraging activity when looking for a path between their colony and a source of food. It was first used to solve the well-known dilemma of the traveling salesman. It is then utilized to solve a variety of difficult optimisation problems. In ACO, a set of software agents called artificial ants search for good solutions for the given optimisation problem. To apply ACO, the optimization problem is first transformed in to the problem of finding the best path of a weighted graph.

Algorithm 3.2 Robot Path Planning using Ant Colony Optimisation

ProcedureACO :
Constructing workspace for robots
Initialize population of ants n,
max limit of iterations I_{max}, *Initial weights X, and target Y*
for *i* 1 *to* I_{max} **do**
  Place each ant in X
  while ant_p *is* $\notin Y$ **do**
    d$\leftarrow$ *list of reachable paths*
  end while
  if *every ant has reached Y* **then**
    $X_p \leftarrow$ *global index of ant p's route*
    $X_{best} \leftarrow$ *best route ofall ants*
    $X_{worst} \leftarrow$ *worst route ofall ants*
  end if
  Choose leader among all the ants
  Using formation control strategy follower antstracking the route of leader ant
  $F_p \in$ *global index of follower ant's path*
end for
Output the best path
End Procedure

Ants are social insects that live in colonies. They dwell in groups called colonies. The behavior of the ants is controlled by the goal of searching for food. While searching, ants roam around their colonies. An ant hops from one location to the next in search of food. It leaves a pheromone-like chemical substance on the ground as it moves. Pheromone trails are used by ants to communicate with one another. When an ant comes across some food, it carries as much as it can. It deposits pheromone on the pathways according on the quantity and quality of the food when it returns. Pheromone can be detected by ants. As a result, additional ants will be able to smell it and will follow that path. The higher the pheromone level, the more likely the ants are to choose that road, and the more ants who follow that path, the more pheromone is produced on that path. In robot path planning the same process is applied, and the artificial ants incrementally build solutions by moving on the graph. The solution construction process is stochastic and is biased by a pheromone model that is a set of parameters associated with graph components whose values are modified at runtime by the ants. While solving the problem the first step generates a solution for each ant, while in the second step paths found

by different ants are compared, and in the last step path value or pheromone are updated.Fig. 1 shows the working flow mechanism of ACO algorithm.

3.3 Glowworm Swarm Optimisation (GSO)

Glowworm swarm optimisation (GSO) is another optimisation technique that can be used for robot path planning. It is inspired by the behaviour of glowworms that use their bioluminescence to attract other glowworms and avoid obstacles. In GSO-based robot path planning, the environment is modelled as a graph, where nodes represent the robot's position and edges represent possible paths between the nodes. The goal is to find the shortest path from the start node to the goal node while avoiding obstacles in the environment.

Algorithm 3.3 Robot Path Planning using Glowworm Swarm Optimisation

Procedure GSO :
Initializing Glowworm Population size P
Define Objective function(x) *and Luciferin Quantity* (L)
for $i = 1$ *to* P **do**
  $L_i(m+1) = (1-\rho)L_i(m) + \gamma Fitness(x_i(m+1))$
  for $j = 1$ *to* i **do**
    $N_j(m) = \{j : d_{ij}(m) < r_d^i(m); L_i(m) < L_j(m)\}$
    $P_{ij}(m) = \frac{L_j(m)-L_i(m)}{\sum_{k \epsilon N_i(m)} L_k(m)-L_i(m)}$
    $X_i(m+1) = X_i(m) + s\left(\frac{X_j(m)-X_i(m)}{\|X_j(m)-X_i(m)\|}\right)$
    $r_i^d(m+1) = min\{r_s, max\{0, r_d^i(m) + \beta(n_m - |N_i(m)|)\}\}$
  end for
end for
End Procedure

The GSO algorithm works by simulating the bioluminescence of glowworms on the graph.Each node in the network is first given a brightness value that indicates how appealing the node is. Nodes close to the start node have their brightness set to a high value, and nodes close to the target node have their brightness set to a low value. After being released from the start node, a bunch of virtual glowworms travel to the nearby node that has the brightest light.

Additionally, a distance function influences the glowworms' movement, favoring

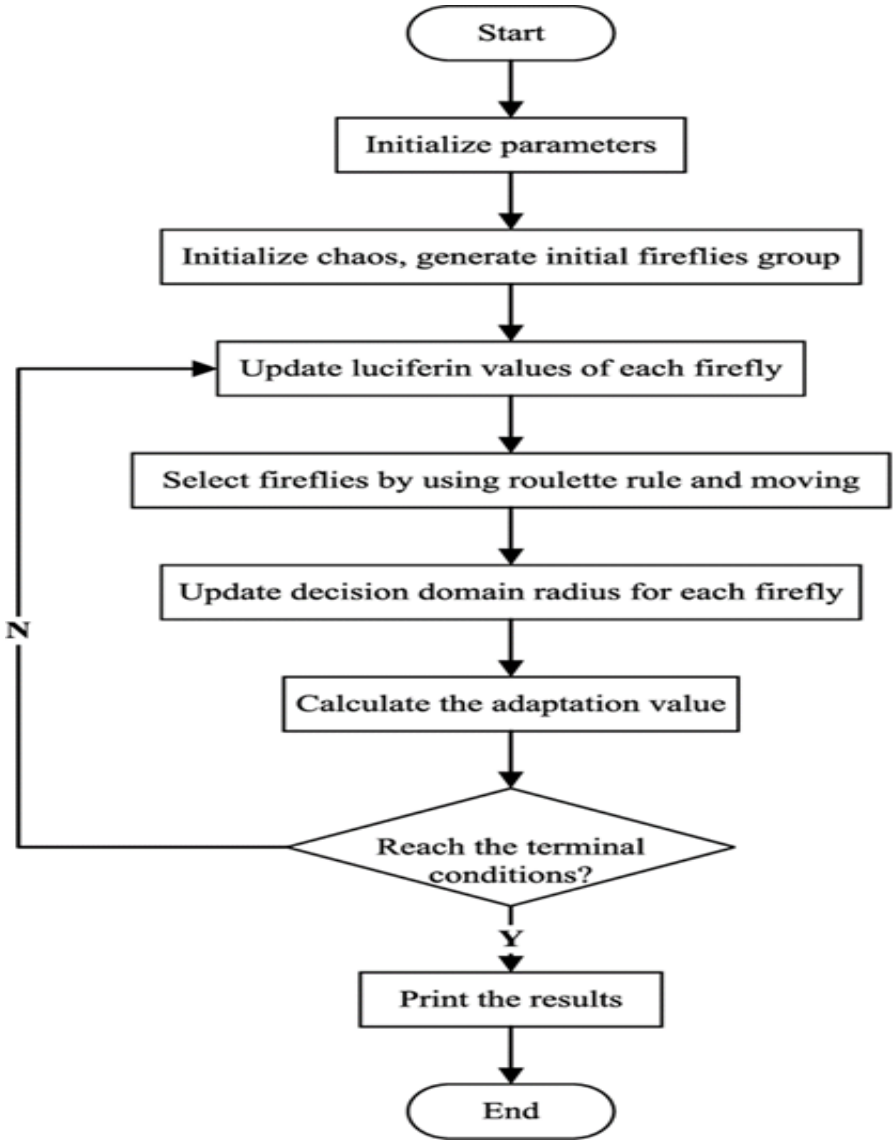

Figure 3.4: Working Flowgraph of GSO

glowworms leaving pheromone behind on the borders of the graph as they go across it. The quality of the glowworm's journey determines how much pheromone is deposited. The pheromone trail is used to update the brightness values of the nodes in the graph. Glowworms can also detect the presence of obstacles and adjust their movements to avoid them. If a glowworm encounters an obstacle, its brightness is reduced, and the brightness of the nodes near the obstacle is also reduced. This encourages other glowworms to avoid the obstacle and find alternative paths.

Once the algorithm converges to a solution, the path with the highest brightness is selected as the optimal path from the start node to the goal node. This path is then used for robot navigation. GSO-based path planning is effective in finding

Table 3.2: Main characteristics of most popular nature-inspired algorithms

S.No	ACO	PSO	GSO
1	Pheromone information used to select region	Net improvement in the objective function at iteration t stored in the global best position Pg. (t)	A glow-worm with the highest luciferin value in the swarm indicates its potential proximity to an optimum
2	Cannot be applied when ants (Agents)have limited sensing range	The direction of movement based on previous best positions	Agent movement along the line of sight with a neighbour
3		The neighbourhood range covers the entire search space	Maximum range is hard-limited by finite sensor range
4	Shifting of selected region's centre in a random direction	Dynamic neighbourhood based on K nearest neighbours (in a local variant of PSO)	Adaptive neighbourhood based on a varying range
5		Limited to numerical optimisation models	Can be applied to multiple sources localisation in addition to numerical optimization

optimal paths in complex environments with multiple obstacles. It is also capable of dynamically adapting to changes in the environment by recalculating the optimal path as the brightness values of the nodes change.

Table 3.3: Comparison of the most frequently used Nature-Inspired algorithms

Characteristics	**ACO**	**PSO**	**GSO**
Year	1992	1995	2005
Author	Marco Dorigo	James Kennedy and Russell Eberhart	K.N.Krishnanand and Debasish Ghose
Optimization	Meta heuristic	Stochastic	Meta heuristic
Parameters	Construct Ant Solutions, Daemon Actions (optional)	Current velocity, Personal Best, Neighbourhood Best	Initialisation, Updating Luciferin, Movement, Updating the Local-Decision Range
Purpose	Find the shortest path	reach the target with minimal duration	Find the local finest solution
Advantages	1)Inherent parallelism, 2) Positive feedback accounts for the rapid discovery of good solutions. 3)Efficient for the Travelling Salesman Problem and similar problems. 4) Can be used in the dynamic application(adapts to changes such as new distances, etc	1) PSO can be applied to both scientific research and engineering use. 2) It has no overlapping and mutation calculation. 3) The search can be carried out by the speed of the particle. 4) PSO adopts the real number code, and it is decided directly by the solution	1) GSO can deal with highly nonlinear, multi-modal optimisation problems naturally and efficiently. 2) GSO does not use velocities, and there is no problem associated with velocity in PSO. 3)The speed of convergence of GSO is very high in the probability of finding the globally optimised answer.
Disadvantages	1) Theoretical analysis is difficult. 2) Sequences of random decisions (not independent). 3) Probability distribution changes by iteration. 4) Research is experimental rather than theoretical.	1) A tendency to a fast and premature convergence in mid-optimum points. 2) The method cannot work out the problems of scattering and optimisation. 3) Slow convergence in a refined search step.	1) GSO is poor in high-dimensional problems. 2)In GSO, the dynamic change of decision domains in the method of glowworms moving, the algorithm slows conventional speed and has poor local search ability delays in the iteration.
Uses	ACO also optimises artificial neural networks for applications in medical image processing.	Detection of Brain tumors using Image segmentation(MRI)	GSO will present new methods for future selection problems.

Chapter 4

Robot Path Planning using Hybrid APGO Approach

Optimization of Robot's Path planning is one of the challenging task being addressed by many researchers in the past, but still it needs to be improvised at a higher level. This paper proposes a novel way of optimizing the robot path planning problem with hybrid nature inspired algorithm. Robot path planning is one of the most complex thing in Artificial Intelligence. And this field evolved a lot since decades. This paper proposes hybrid combination of three nature inspired algorithms to optimize robot path planning, ACO (Ant colony optimizer), PSO (Particle swarm optimizer), GSO (Glowworm search optimizer) are implemented for Robot path planning. The reason to use this three different algorithms is that, in GA they use discrete optimizer for finding the best among all, while in ACO they use meta heuristic optimization for finding shortest path and in PSO it implements stochastic optimization which is used to reach target with minimal duration and in GSO again they use meta heuristic optimization for finding the local finest solution. Since, each optimization technique has different approach and different purpose. And each one has its own advantage and disadvantage. So, the idea is to implement them in hybrid form to get benefit of each optimization. The concept of APGO is presented in this work in terms of path length and path time. This paper proposes the hybrid mechanism of ACO, PSO

and GSO algorithm to solve the path planning problem. The result is compared and presented in the paper which outperforms when compared with the traditional approaches.

4.1 Introduction

Here in this section the proposed methodology of hybrid nature inspired algorithm is presented for solving the problem of robot path planning. Algorithm. 1 depicted shows the fundamental mechanisms opted for deriving optimized path for robots. As described in the previous section the traditional ACO works in a fixed multidirectional way usually in a eight directions, due to which it faces many problems like it requires more number of iterations, multiple corner problems, increase in energy and time requirement. Thus it overall effects the performance of the solution. While combining it with PSO it gives better results in terms of its performance. In the hybrid approach of ACO-PSO when the ACO algorithm is applied for finding the solution for robots while moving from its source coordinates to destination coordinate, it stucks at a point when it encounters the obstacle in its path, this is the point where the algorithm is required to take a backstep from its position and look for some other movement which can avoid the collision, this is where the PSO helps them in reducing the number of iteration, it provides the list of available moves along with the coordinates having minimum value of the objective function. This makes the approach more efficient in selecting the next node to reach destination in a much faster way.

4.2 Proposed Hybrid Approach

In this research, the hybrid approach was taken to another level by adding some feature of GSO with this hybrid approach. The GSO algorithm works on the fundamental of foraging behavior of lightening worms known as glow worms. While selecting nodes based on the minimum value of fitness function out of the available

moves for robot to select when stuck at one place. Just like other conventional approaches which locates global solution in the objective domain, this algorithms works by dividing its swarm of population in disjoint groups which converge at different points containing the local optima to find multiple solutions. Therefore combining GSO with the hybrid approach of ACO-PSO accelerates the procedure of selecting nodes, as it converges at more pace and and selects nodes from all the present node which adds to its importance and improves the performance to another level for robot path planning.

Algorithm 4.4 Path Planning

Require: Time taken
Ensure: Traversal
1: *Initialize* S_{xy}, G_{xy}
2: *Sense for obstacles* O_{xy} in E
3: **if** $O \in SG$ **then**
4: *implement RCSA*
5: **end if**
6: r proceeds towards G
7: **if** $r == G$ **then**
8: *Stop*
9: **end if**
10: Or Else *GOTO step* 2

Figure 4.1: Different stages showing the working of Path Finding for Robots

4.3 Results and Discussion

This section presents the result obtained by experimenting the proposed approach. This paper explored the approach of hybrid mechanism of nature inspired algorithms. The result obtained is presented in both tabular and graphical from in this section. Here in this paper we have considered three most well known algorithm for experimentation. They are first tested individually and then the selective behavior of each one is combined with the other and then again tested it in the same scenario. The upshoot in the performance encourgaed to again make hybrid combination of

three algorithms which is termed as APGO in this paper. The final result which came out from APGO is presented in the paper which is having a competitve edge over the other combination in terms of path length and path time. To approximate the shortest path situations, like- in maps, where there can be many hindrances. Our First Basic Map with 2D Grid having several obstacles and we start from a source cell (colored Green below) to reach towards a goal cell which denote as Red, Obstacles is Represent by Using Black Color, We are Calculation Approximate the value of point using some heuristics which consume less time Pre-compute the distance between each pair of cells using Euclidean Formula. It is Simply but the sum of absolute values of differences in the two point in cell. Fig. 4 presents the 04 different instances of the experiment carried out, Fig. 5 present in this section is another instance of experiment. The result obtained are presented in Table. 1 and the graphical comparision is presented in Fig. 6 and Fig. 7. The paper focuseed on the hybrid mechanism for robot path planning, to accoomplish the hybrid combination the approach mention in Eq. 1 - Eq. 12 are considered during amalgamation. Eq. 1 - Eq. 5 represents the approach of GSO, Eq. 6 - Eq. 7 represents the PSO approach and EQ. 8- Eq. 9 presents GSO approach. To connect all the nodes euclidian distance approach mentioned in Eq. 10 - Eq.12 is considered.

4.3.1 GSO Approach

$$l_i(t+1) = (1-\rho)l_i(t) + \gamma Fitness(x_i(t+1)) \tag{4.1}$$

$$N_i(t) = \left\{ j : d_{ij}(t) < r_d^i(t); l_i(t) < l_j(t) \right\} \tag{4.2}$$

$$P_{ij}(t) = \frac{l_j(t) - l_i(t)}{\sum_{k \epsilon N_i(t)} l_k(t) - l_i(t)} \tag{4.3}$$

$$X_i(t+1) = X_i(t) + s\left(\frac{X_j(t) - X_i(t)}{\|X_j(t) - X_i(t)\|}\right) \tag{4.4}$$

$$r_i^d(t+1) = min\left\{r_s, max\left\{0, r_d^i(t) + \beta\left(n_t - |N_i(t)|\right)\right\}\right\} \tag{4.5}$$

4.3.2 PSO Search Strategy

$$x_i^{t+1} = x_i^t + v_i^t * t \tag{4.6}$$

$$v_i^{k+1} = wv_k^i + c_1 r_1\left(xBest_i^t - x_i^t\right) + c_2 r_2\left(gBest_i^t - x_i^t\right) \tag{4.7}$$

4.3.3 ACO Approach

$$P_{ij}^k = \frac{\left(\tau_{ij}^{\alpha}\right)\left(\eta_{ij}{}^{\beta}\right)}{\sum_{z\epsilon allowed_i}\left(\tau_{ij}^{\alpha}\right)\left(\eta_{ij}{}^{\beta}\right)} \tag{4.8}$$

$$\tau_{ij} \leftarrow (1-\rho)\tau_{ij} + \sum_{k}^{m} \Delta\tau_{ij}^k \tag{4.9}$$

$$Distance = \mid (cell.x - end.x) \mid + \mid (cell.y - end.y) \mid \tag{4.10}$$

Here when we Move from top right left up down direction using diagonal distance, it is calculated by using the following formula.

$$x = \mid (x - end.x) \tag{4.11}$$

$$y = \mid (y - end.y) \tag{4.12}$$

This section presents the results of the experiment performed for the problem of robot path planning in five different reference maps. The Table. 2 presents the results obtained from the traditional approaches like ACO, PSO and GSO in terms of path length and path time. The experiment carried out on 05 different reference maps and the results obtained are then compared with the proposed hybrid approaches. Here in this paper we proposed the hybrid nature inspired algorithms

for robot path planning. The combination of above mentioned three algorithms when applied to the reference maps outperforms in terms of path length and path time. The results are also compared with 02 another hybrid approach. The combination of ACO is done with both PSO and GSO which also performs better than the traditional approach but the APGO gives much better results. Inducing GSO with ACO-PSO paced the approach and outperformed as compared to the other combination. The hybrid combination of algorithms perform both an exploratory and exploitative search, unlike the existing traditional approaches which are biased, towards either an exploitative search or an exploratory search. The results in terms of path length and path time is presented in the paper which clearly shows a great improvement in terms of performance for this specific problem.

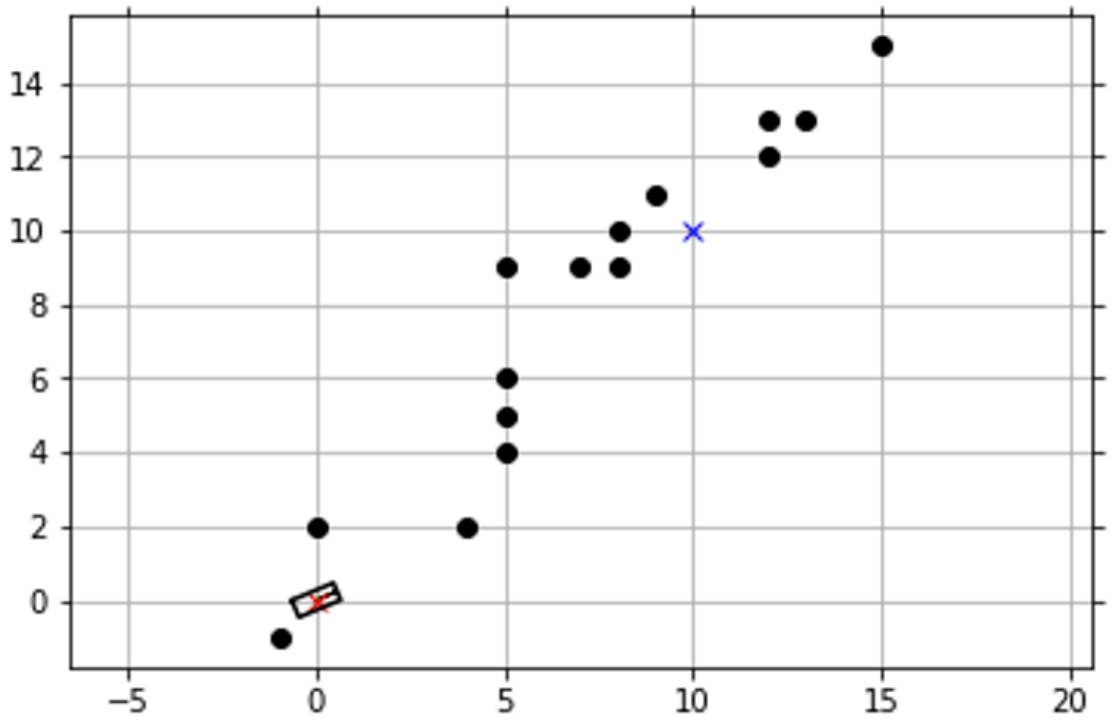

Figure 4.2: Instance of algorithm deriving optimized path for robot

4.4 Conclusion

In this chapter, we proposed a hybrid nature inspired algorithms for robot path planning. The hybrid combination APGO presented in this paper outperformed the

Table 4.1: Comparison of results obtained for the path length on the four different reference maps using the traditional nature inspired algorithms and proposed hybrid approach

Reference Map	**Map Size**	**ACO**	**PSO**	**GSO**	**ACO+GSO**	**ACO+PSO**	**APGO**
Map 1	500*500	102.12	127.56	112.77	87.76	89.92	81.11
Map 2	500*500	113.11	150.96	156.23	85.52	87.01	80.55
Map 3	500*500	119.09	149.55	152.45	98.92	107.01	91.12
Map 4	500*500	131.55	151.99	171.32	101.12	108.23	100.71
Map 5	500*500	152.53	183.12	177.85	110.98	119.95	109.91

Table 4.2: Comparison of results obtained for the path time on the four different reference maps using the traditional nature inspired algorithms and proposed hybrid approach

Reference Map	**Map Size**	**ACO**	**PSO**	**GSO**	**ACO+GSO**	**ACO+PSO**	**APGO**
Map 1	500*500	109.12	127.06	107.05	66.78	66.75	22.92
Map 2	500*500	118.56	140.42	115.32	75.09	76.65	29.11
Map 3	500*500	135.44	156.08	119.89	79.06	79.98	33.67
Map 4	500*500	129.76	151.27	119.16	80.02	81.19	33.02
Map 5	500*500	151.87	172.73	121.23	85.56	86.02	37.78

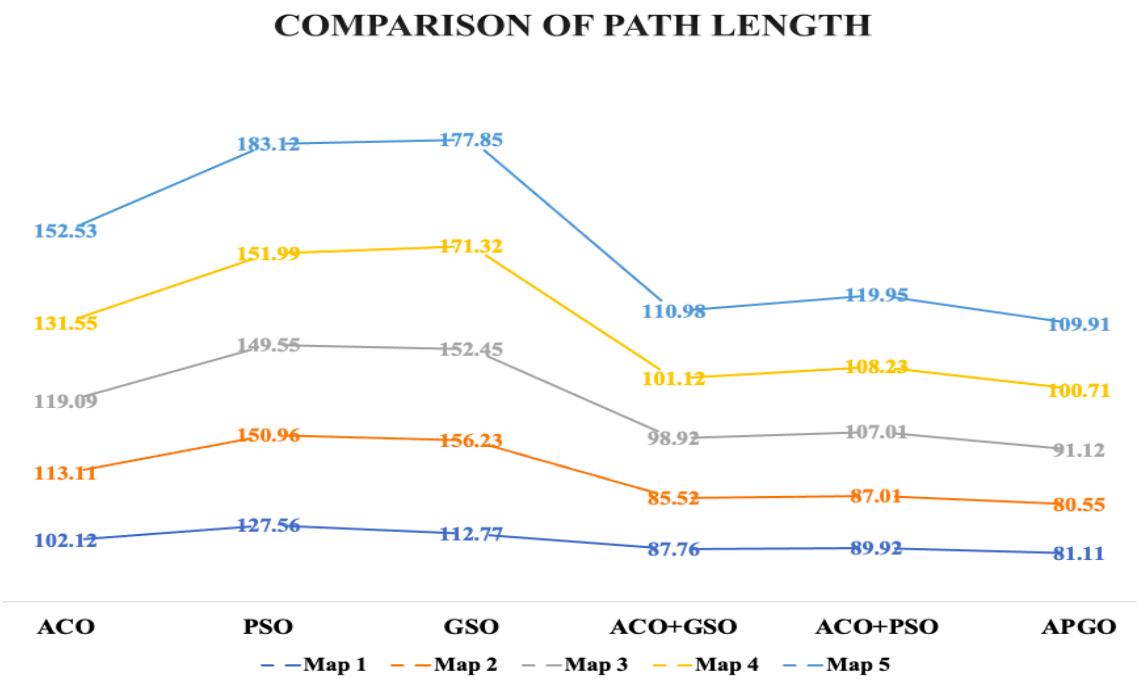

Figure 4.3: Comparison of Path Length

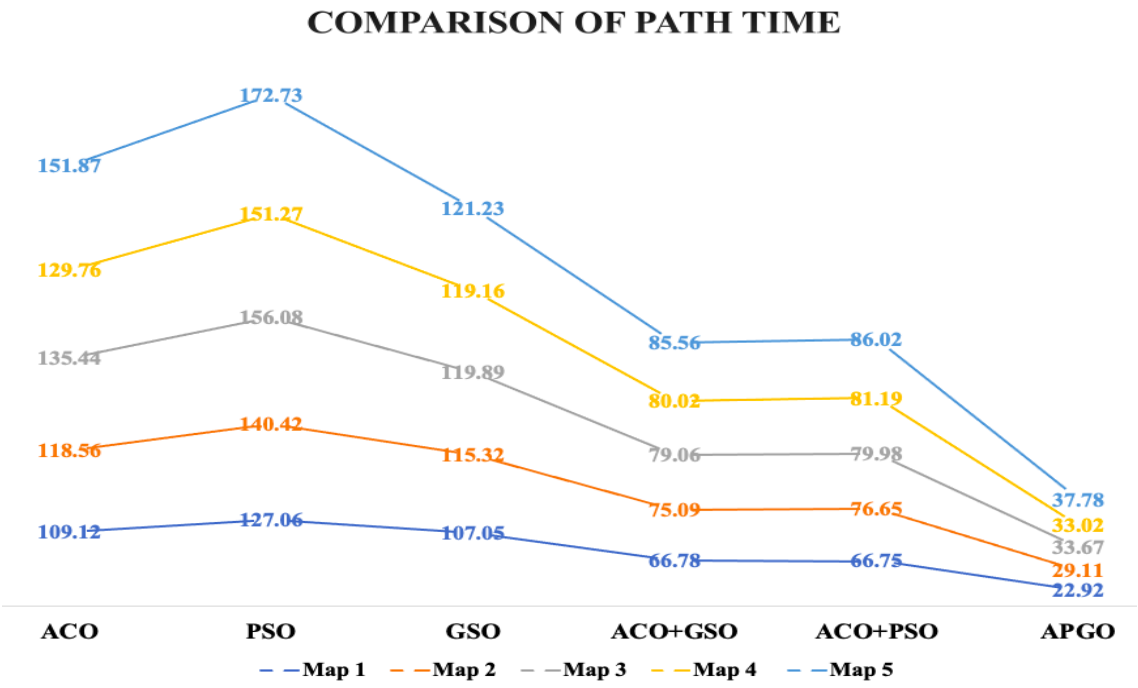

Figure 4.4: Comparison of Path Time

other combination presented in the paper as well as it also produced better results than the traditional algorithms when operated individually. The result presented in this paper progressed gradually, first we tested only single algorithms, then we made a combination of two algorithms and finally then we combined the three algorithms. The involvement of GSO algorithm within the combination of ACO and PSO made

the node selection behavior faster and efficient. The path length and path time parameters are considered for the validation of results. Both the parameters resulted in a very efficient way. The future extension of this work could be to test this hybrid approach to the dynamic environment where the nature of obstacles are moving and having multiple robots in the environment.

4.4.0.1 Results and Discussions

Chapter 5

Multi-Agent Path Finding Algorithms

5.1 Introduction

Path planning is the process of finding an optimal or near-optimal path between two points in a given environment while avoiding obstacles and other constraints. It is a significant issue in a number of domains, such as video games, computer graphics, robotics, and driverless cars. Finding a workable route that meets the following requirements is the aim of path planning: it must connect the starting point and the destination point, avoid obstructions and other environmental constraints, maximize a particular performance metric like distance, time, or energy, and account for the environment's dynamic nature and any changes that may arise during execution [1]. Depending on their methodology and the kind of problem they address, path planning algorithms can be divided into several groups [1]. Finding the best or nearly best route between two points in a given environment while avoiding obstructions and other limitations is the goal of path planning, a crucial problem-solving approach [2]. Automated warehouses, intersection control, airport surface operations, automated parking, and video games are just a few of the many possible uses for path planning [1].] Path-planning algorithms are used in these applications

Figure 5.1: Core-strategies in MAPF

to provide safe and efficient navigation, generate realistic and natural-looking motion, and enable users to navigate through virtual environments. The development of effective path-planning algorithms has the potential to significantly improve the performance, efficiency, and safety of various autonomous systems and applications. The major problem in multi-agent path planning methods is the complexity of the problem as the number of agents increases. In multi-agent path planning, multiple agents need to navigate through the same environment simultaneously, and the paths of different agents may interfere with each other. This interference can lead to conflicts such as collisions or deadlock situations, which need to be resolved to find a feasible path for all agents. The complexity of the problem and the coordination and communication requirements make multi-agent path planning a challenging problem to solve, and research is ongoing to develop effective algorithms and techniques to address these challenges. Conversely, it is possible to generate solutions that are not optimal but can be obtained quickly (6 to 10). These solutions may not prioritize quality but are still preferable to having no solution at all due to waiting for optimal solvers that may not meet the deadline. Optimal solvers cannot guarantee to find solutions within a specified time frame. By obtaining workable solutions rapidly, we can utilize the remaining time until the deadlines to perform iterative improvement. This is the fundamental concept behind anytime algorithms which is to produce a feasible solution when interrupted, and whose quality will increase with time. In our research, we have investigated several algorithms to obtain the initial solution, such as Conflict-Based Search (CBS), Improved Conflict-Based Search (ICBS), Priority

Inheritance with Backtracking (PIBT), Hierarchical Cooperative A* (HCA*), and Windowed Hierarchical Cooperative A* (WHCA*). Our study involves a thorough comparison of these methods based on their ability to provide a collision-free path. We have evaluated these methods and recommended the most suitable approach to achieve an optimal solution. Although iterative refinement is a promising approach for Multi-Agent Path Finding (MAPF), it has been underutilized due to the challenge of incrementally improving a known solution. In the context of local

search, this involves identifying a favorable neighborhood solution. Therefore, we introduce an anytime framework of iterative refinement for MAPF[3]. Our research paper focuses on comparing various path-planning algorithms used for finding an initial path and optimising it. Path planning algorithms are essential techniques used to find an optimal or near-optimal path between two points while avoiding obstacles and other constraints. In our study, we explore the effectiveness of different path-planning algorithms in finding an initial path, which is then optimised through iterative refinement. By comparing and evaluating the performance of these algorithms, we aim to determine the most efficient and effective method to achieve an optimal path for various applications such as robotics, autonomous vehicles, and computer graphics.

5.2 Methodology

In order to create an anytime version of the optimal MAPF algorithm, Standley and Korf . expanded on their earlier work. A form of A-based anytime algorithm was investigated by Cohen et al. and utilised in MAPF. An anytime MAPF solver, X assumes sparse circumstances, i.e., sparsely dispersed agents in fields with low collision probability. These three techniques iteratively tighten the constraints to eventually get optimal solutions after initially searching for non-optimal answers by loosening some constraints. They are each dependent on a particular solver, which has the disadvantage that they might not find initial solutions within a reasonable amount of time, returning nothing.Surynek researched the neighbourhood repair rules for a pebble motion on graphs;It is MAPF-adaptable. Improvements are made through ad hoc local adjustments, therefore the solution still contains redundant instances of a priori unidentified patterns.

5.3 Related Contributions

5.3.1 Hierarchical Cooperative A* (HCA)

The use of heuristics based on abstractions of the state space is a common approach to improving pathfinding algorithms. There are two generic methods for improving such heuristics[30]. The first involves precomputing all distances in the abstract space and storing them in a pattern database, but this is not feasible for dynamic maps such as those encountered in Cooperative Pathfinding. The second approach is to use Hierarchical A*, where abstract distances are computed on demand. This is more suitable for dynamic environments.

In 2017, Yu et al. proposed an extension to the HCA* algorithm called Weighted Hierarchical Cooperative A* (WHCA*) to address the issue of suboptimal solutions. The authors introduced a weight factor to the heuristic function used in HCA* to improve the quality of the obtained solution[30].

When testing HCA* and WHCA* In maze-like environments, research has shown that the success rate of all methods is nearly the same when the number of agents is small. However, as the number of agents increases, the success rate begins to decrease for some algorithms. WHCA* has been found to have the best success rate compared to other methods[5].

5.3.2 Windowed Hierarchical Cooperative A* - (WHCA)

WHCA* is an algorithm used for multi-agent path planning in robotics. It is designed to coordinate the movements of multiple agents in an environment with the goal of reaching their respective destinations while avoiding collisions.The algorithm operates in a hierarchical manner, consisting of high-level and low-level planning stages. In the high-level stage, agents plan their paths within a specific time window. The time window represents a discrete period during which the agents' paths are planned. The high-level planner takes into account the current and predicted positions of other agents to determine a collision-free path for each agent within its

assigned time window.

In the low-level stage, a local planner, typically using the A* algorithm, is employed to generate the detailed trajectory for each agent within its time window. The local planner considers the current state of the environment, including the positions of other agents, to avoid collisions while following the planned high-level path..

WHCA* is known for its ability to handle complex scenarios with multiple agents and dynamic obstacles. It provides a centralized approach for coordination and collision avoidance, ensuring that agents can navigate safely while achieving their individual goals.This algorithm has found applications in various robotic domains, including autonomous vehicles, swarm robotics, and multi-robot systems in industrial settings. Its hierarchical and cooperative nature makes it well-suited for scenarios where multiple agents need to navigate in a coordinated manner while adhering to safety constraints.

5.3.3 Conflict Based Search (CBS)

In 2005, Sharon et al. presented Conflict-Based Search (CBS), an algorithmic paradigm for resolving the multi-agent pathfinding (MAPF) problem. To increase its scalability and effectiveness, CBS has undergone various upgrades and changes since its debut[6]. Prioritised Planning (PP), an early addition to CBS that was suggested by Silver in 2005, prioritizes the search's limits. PP enabled CBS to resolve more significant and intricate MAPF issues.The addition of Independence Detection (ID), which was suggested by Sharon et al. in 2008, was another enhancement to CBS. In order to narrow the search space and improve the scalability of CBS, ID takes advantage of the independence amongst agents[7].Enhanced Conflict-Based Search (ECBS)[8], which presented a new method for handling conflicts in the search, was proposed by Sharon et al. in 2009. The dispute resolution policy utilized by ECBS is more adaptable than the one employed by CBS, enabling more efficient search in challenging circumstances.Later, in 2011, Boyarski and Sharon put out the Generalized-Screening CBS (GS-CBS) form of CBS, which makes use of a cutting-

edge pruning method based on unfavorable solutions to increase scalability.Boyarski et al. presented the idea of incremental search to CBS in 2012[7]. The process of incremental search entails breaking down the search space into smaller subproblems and addressing each one one at a time. CBS is able to manage more substantial and challenging issues because of this method.Since its debut, CBS has evolved significantly, and many changes and upgrades have been suggested to increase its scalability and effectiveness. These adjustments have made it possible for CBS to tackle larger and more intricate MAPF issues and have prompted the creation of a number of algorithmic variations that specifically target different difficulties in MAPF[7].

5.3.4 Improved Conflict-Based Search Algorithm for Multi-Agent Pathfinding(ICBS)

The research paper titled "ICBS: Improved Conflict-Based Search Algorithm for Multi-Agent Pathfinding" by Eli Boyarski proposes an improved version of the Conflict-Based Search (CBS) algorithm for Multi-Agent Pathfinding (MAPF) problems. Although the CBS algorithm is a popular approach for solving MAPF problems, it has some limitations regarding scalability and efficiency[44].

The proposed ICBS algorithm addresses these limitations by introducing modifications such as the use of a more efficient priority queue data structure to store the search nodes, a novel heuristic function that considers the cost of future conflicts, and a dynamic threshold strategy to adaptively adjust the search space. The experimental results show that the ICBS algorithm outperforms the original CBS algorithm and other state-of-the-art MAPF algorithms in terms of solution quality and computation time on various benchmark instances. The ICBS algorithm also scales better with the number of agents and the size of the environment, making it suitable for solving large-scale MAPF problems. In summary, the ICBS algorithm is a significant improvement over the CBS algorithm for solving MAPF problems and is a valuable contribution to the field of multi-agent systems and robotics. One

of the significant improvements of ICBS is the use of an efficient priority queue data structure that reduces the number of nodes requiring expansion, resulting in a significant reduction in computation time. The proposed algorithm's novel heuristic function helps to guide the search process more effectively by focusing on the most promising regions of the search space[8]. ICBS introduces a dynamic threshold strategy to adaptively adjust the search space, allowing the algorithm to focus on the most promising regions and ignore less favorable areas. This improves the scalability and reduces computation time. The research paper demonstrates through experimental results that ICBS surpasses the original CBS algorithm and other MAPF algorithms in terms of solution quality and computation time on a variety of benchmark instances. ICBS scales better with the number of agents and the environment size, making it suitable for large-scale MAPF problems. Consequently, the proposed ICBS algorithm is a significant improvement over the CBS algorithm, making it a valuable contribution to multi-agent systems and robotics. ICBS can be applied in various applications such as warehouse automation, traffic control, and unmanned aerial vehicle (UAV) coordination. The paper presents an enhanced version of ICBS by Eli Boyarski that addresses issues with the current implementation of ICBS, including cases where it fails to find a solution or takes a long time. Multi-Agent Pathfinding (MAPF) is an important problem in the field of robotics and artificial intelligence, where multiple agents must navigate through a shared environment without colliding with each other. MAPF can be applied in various domains, such as warehouse automation, traffic control, and unmanned aerial vehicle coordination.The Improved Conflict-Based Search (ICBS) algorithm is a popular approach to solving MAPF problems.

However, ICBS has some limitations, such as being stuck in certain situations when there are many agents and few paths available. In a recent paper, Eli Boyarski proposed several improvements to the ICBS algorithm to address these limitations.One of the improvements proposed is prioritized planning, which involves planning agents in a certain order based on a heuristic. This helps to reduce the num-

ber of conflicts that need to be resolved, as agents planned first can avoid conflicts that later agents would encounter. Another improvement is independent conflict avoidance, which divides agents into groups and plans their routes independently. This can help to avoid conflicts that would be difficult to resolve otherwise.

The third improvement is the delayed conflict resolution strategy, which involves delaying the resolution of conflicts until they become necessary, rather than resolving them immediately.

This can help to reduce the number of conflicts that need to be resolved overall, as some conflicts may resolve themselves without needing explicit intervention. The fourth improvement is the expansion conflict detector, which detects conflicts during the expansion phase of the search process, rather than during the conflict detection phase. This helps to reduce the search space by avoiding unnecessary branches that lead to conflicts.To evaluate the effectiveness of the enhanced ICBS algorithm, Boyarski conducted a series of experiments on standard MAPF benchmarks. The results showed that the enhanced algorithm outperformed the original ICBS algorithm in terms of both solution quality and runtime in all cases. The largest improvement was observed in cases where there were many agents and few paths available, where the enhanced algorithm could find solutions up to five times faster than the original.In conclusion, Boyarski's proposed improvements to the ICBS algorithm are a significant contribution to the field of multi-agent systems and robotics. The enhanced ICBS algorithm is more efficient and effective in solving MAPF problems, making it suitable for large-scale applications.

5.3.5 Priority Inheritance with Backtracking (PIBT)

PIBT works by assigning priorities to each agent based on their position in the environment or other criteria. The algorithm then attempts to move the agents along their optimal paths while avoiding collisions with other agents[2]. If a collision is detected, PIBT employs a backtracking mechanism to temporarily suspend the lower-priority agent and allow the higher-priority agent to move. The backtrack-

ing mechanism in PIBT involves temporarily suspending the lower-priority agent and attempting to move the higher-priority agent again. This process continues until all agents have reached their destinations without any collisions. The result is a provably correct and optimal solution to the MAPF problem[2]. PIBT has several advantages over other MAPF algorithms, including the ability to handle large and complex environments, the ability to generate provably correct and optimal solutions, and the ability to minimize communication overhead between agents. However, PIBT may not be suitable for scenarios where agents have incomplete or inconsistent knowledge of the environment, or where the robustness of the solution is a critical requirement.

In summary, PIBT is a powerful algorithmic approach for solving MAPF problems that prioritizes the generation of[2] provably correct and optimal solutions, making it a suitable choice for scenarios where these criteria are critical requirements.

5.4 Comparison

5.4.1 A*, HCA*, and WHCA*

Several studies have shown that when a series of challenging, maze-like environments consisting of 32x32 4-connected grids with 20 % randomly placed impassable obstacles and filled disconnected subregions were tested with different algorithms, all methods achieved a 100% success rate with a small number of agents. However, when more agents were added to the map, Local Repair A* was not able to cope with the congestion, resulting in 20% of the agents failing to reach their destinations due to a bottleneck. On the other hand, the three Cooperative A* algorithms allowed agents to navigate through bottlenecks by stepping aside and allowing others to pass, resulting in a higher success rate even with a larger number of agents. Among these algorithms, WHCA* with a window size of 16 (indicated in parentheses in the figures) achieved the best success rate, with less than 2% of the agents

failing to reach their destination, making it a more efficient and effective approach for pathfinding in crowded environments.

5.4.2 WHCA better than HCA

One issue with the previous algorithms is how they terminate once the agents reach their destination. If an agent sits on its destination, for example in a narrow corridor, then it may block off parts of the map to other agents. Ideally, agents should continue to cooperate after reaching their destinations, so that an agent can move off its destination and allow others to pass. A second issue is the sensitivity to agent ordering. Although it is sometimes possible to prioritise agents globally (Latombe 1991), a more robust solution is to dynamically vary the agent order, so that every agent will have the highest priority for a short period of time. Solutions can then be found which would be unsolvable with an arbitrary, fixed agent order. Thirdly, the previous algorithms must calculate a complete route to the destination in a large, three-dimensional state

space. With single agent searches, planning and plan execution are often interleaved to achieve greater efficiency (see for example Korf's Real-Time Heuristic Search (1990)), by avoiding the need to plan for long-term contingencies that do not in fact occur. WHCA* develops a similar idea for cooperative search. A simple solution to all of these issues is to window the search. The cooperative search is limited to a fixed depth specified by the current window. Each agent searches for a partial route to its destination, and then begins following the route. At regular intervals (e.g. when an agent is half-way through its partial route) the window is shifted forwards and a new partial route is computed. To ensure that the agent heads in the correct direction, only the cooperative search depth is limited to a fixed depth, whilst the abstract search is executed to full depth. A window of size w can be viewed as an intermediate abstraction that is equivalent to the base level state space for w steps, and then equivalent to the abstract level state space for the remainder of the search. In other words, other agents are only considered for

w steps (via the reservation table) and are ignored for the remainder of the search. To search this new search space efficiently, a simple trick can be used. Once w steps have elapsed, agents are ignored and the search space becomes identical to the abstract search space. This means that the abstract distance provides the same information as completing the search. For each node Ni reached after w steps a special terminal edge is introduced, going directly from Ni to the destination G, with a cost equal to the abstract distance from Ni to G. Using this trick, the search is reduced to a w-step window using the abstract distance heuristic introduced for HCA*. In addition, the windowed search can continue once the agent has reached its destination. The agent's goal is no longer to reach the destination, but to complete the window via a terminal edge. Any sequence of w moves will thus reach the goal. However, the WHCA* search will efficiently find the lowest cost sequence. This optimal sequence represents the partial route that will take the agent closest to its destination, and once there to stay on the destination for as much time as possible Experiment result shows that WHCA takes 9 times to complete the path which is more than less tha HCA but more than PIBT. It has less Sum of cost is more in WHCA compared to HCA.

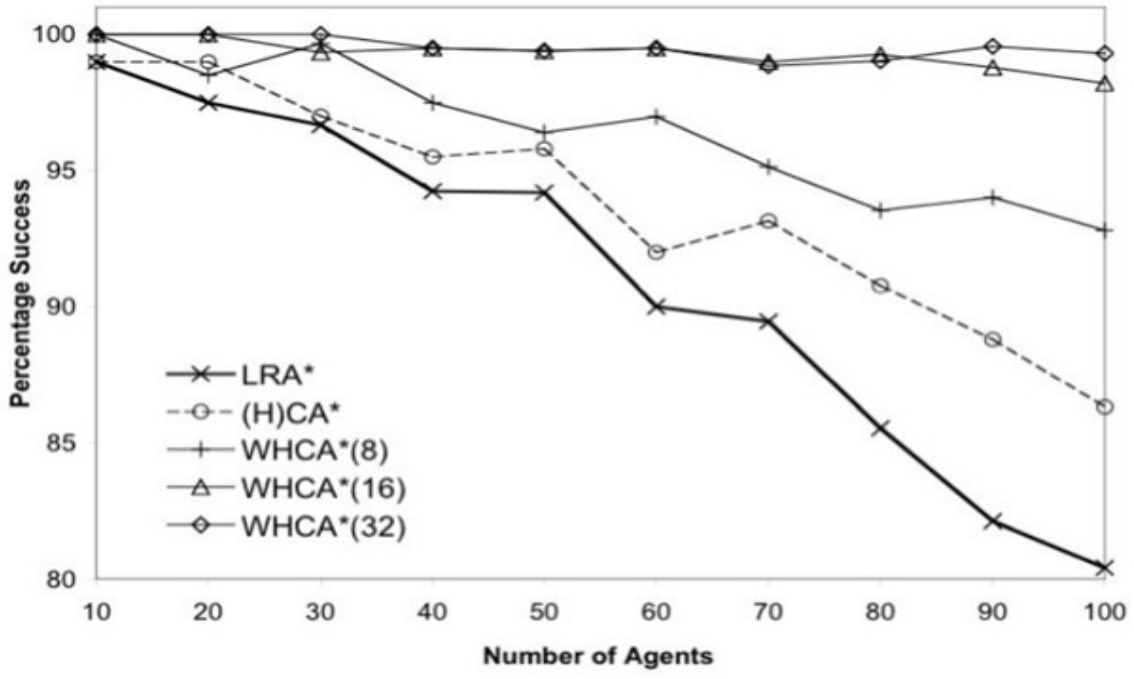

Figure 5.2: Comparison of WHCA , LRA ,H(CA)*

5.4.3 Comparison between PIBT,CBS and ICBS

One of the key advantages of PIBT over CBS and ICBS is its ability to handle multiple types of constraints, such as temporal and energy constraints, in addition to the traditional collision avoidance constraints. This is due to its ability to use a priority function that takes into account multiple criteria, allowing for more flexible and efficient path planning.

Another advantage of PIBT is its ability to handle high levels of uncertainty in the environment, such as incomplete information about other agents' goals or movements. PIBT can handle this uncertainty by using a backtracking mechanism that allows agents to revise their plans when new information becomes available.

In terms of computational complexity, PIBT has a worst-case time complexity of $O(n\hat{3})$, where n is the number of agents, while CBS and ICBS have a worst-case time complexity of $O(b\hat{n})$, where b is the branching factor of the search tree. This makes PIBT a more scalable and efficient approach for large-scale multi-agent pathfinding problems.Finally, PIBT has been shown to outperform CBS and ICBS in several benchmarks and real-world scenarios, including scenarios with dynamic obstacles and multiple types of constraints.

5.4.4 Comparison between PIBT , HCA* and WHCA*

Compared to HCA* and WHCA* , PIBT assumes that agents have complete and consistent knowledge of the environment, which allows it to generate optimal solutions that are provably correct. In contrast, HCA* and WHCA* assume that agents have incomplete and inconsistent knowledge, which makes it challenging to guarantee optimality or correctness.

Moreover, PIBT can handle complex planning problems where agents have conflicting goals, as it can resolve conflicts by assigning priorities based on the goals of the agents involved. This is not the case with HCA*, which generates subtasks for agents without considering their goals or priorities, potentially leading to suboptimal or inefficient solutions. Additionally, PIBT can generate solutions that are

not only optimal but also robust to changes in the environment, while HCA* and WHCA* may require significant replanning when the environment changes.

However, PIBT is computationally expensive, especially for problems with a large number of agents or complex temporal constraints. To improve the algorithm's efficiency, PIBT can be optimized using techniques such as heuristic search and pruning. Overall, PIBT is a suitable choice for problems that require optimal solutions and intricate planning, but may not be the best choice when there is a need for agents to cooperate and share information to achieve their goals, or when the environment is subject to frequent changes that require dynamic replanning.

5.4.5 Comparison between ICBS and PIBT

The Improved Conflict-Based Search Algorithm for Multi-Agent Pathfinding (ICBS) and Priority Inheritance with Backtracking (PIBT) are two popular approaches for solving the Multi-Agent Pathfinding (MAPF) problem. While ICBS is an improved version of the Conflict-Based Search (CBS) algorithm, PIBT is a method that uses priority inheritance and backtracking to solve MAPF problems.Both ICBS and PIBT have been shown to be effective in solving a variety of MAPF problems. ICBS has been shown to be effective in finding high-quality solutions quickly, while PIBT has been shown to be effective in solving problems with large numbers of agents.

In terms of solution quality, ICBS has been shown to outperform the original CBS algorithm and other state-of-the-art MAPF algorithms, including PIBT, on a variety of benchmark instances. However, PIBT has been shown to be effective in solving large-scale MAPF problems, where ICBS may not be as effective due to its high computational requirements.One key difference between ICBS and PIBT is the use of priority inheritance in PIBT. This technique allows agents with higher priority to preempt agents with lower priority in order to avoid conflicts. This can be particularly effective in situations where there are many agents with similar start and goal locations. In contrast, ICBS uses a conflict-based approach that focuses on resolving conflicts between agents by delaying their movements and assigning higher

priority to agents that are involved in more conflicts.

Another key difference between ICBS and PIBT is the use of backtracking in PIBT. This technique allows agents to backtrack and re-plan their paths in order to avoid conflicts. This can be particularly effective in situations where agents need to make decisions based on uncertain or changing information. In contrast, ICBS uses a heuristic function to guide the search and prioritize the search space based on the likelihood of finding a solution.In summary, the choice between ICBS and PIBT depends on the specific requirements of the MAPF problem at hand. If finding high-quality solutions quickly is the priority, ICBS may be a better choice. If solving problems with large numbers of agents or uncertain information is the priority, PIBT may be a better choice due to its use of priority inheritance and backtracking techniques.

5.4.6 Comparison between ICBS and WHCA

Two well-liked methods for resolving the multi-agent pathfinding (MAPF) problem are Windowed Hierarchical Cooperative A* (WHCA*) and Improved Conflict-Based Search Algorithm for Multi-Agent Pathfinding (ICBS).The Conflict-Based Search (CBS) method has been enhanced, and the ICBS algorithm is intended to identify high-quality solutions for a variety of MAPF problems efficiently. In contrast to CBS, the algorithm makes a number of changes that improve its scalability and efficiency. These changes include the introduction of a novel heuristic function, a more effective priority queue data structure, and a dynamic threshold technique.

On the other hand, WHCA* is a centralised method that addresses MAPF issues via a hierarchical cooperative search technique. The method breaks the issue down into more manageable subproblems, which are then collectively solved in a hierarchical fashion. In order to detect potential collisions and avoid them in real-time, WHCA* also employs a window-based method.On a variety of benchmark situations, it has been demonstrated that ICBS performs better than the original CBS algorithm and other cutting-edge MAPF algorithms in terms of solution quality and

computation time. Additionally, it has been demonstrated that ICBS scales better with the quantity of agents and the size of the environment, making it appropriate for handling complex MAPF issues.

It has been demonstrated that WHCA* works well in real-world circumstances and can identify workable solutions for a large number of agents quickly. However, the quality of the solutions might not always be the best, and the algorithm might have trouble handling more complicated issues.A 2017 research titled "Comparing Cooperative Pathfinding Algorithms: A Study on Large-Scale Multi-Agent Systems" conducted a comparison between ICBS and WHCA*. On a range of benchmark examples, the study discovered that ICBS beat WHCA* in terms of solution quality, scalability, and computational efficiency.

Chapter 6

Dynamic Hurdle Explainable AI

In recent years, the increasing adoption of multi-robot systems (MRS) has transformed automation across domains such as smart factories, automated warehouses, drone-based surveil- lance, autonomous transportation, and robotic exploration. As these systems grow in both agent count and operational complexity, effective coordination, path planning, and conflict resolution have emerged as critical research challenges. A core issue in these environments is the Multi-Robot Path Planning (MRPP) problem, which involves computing safe, efficient, and collision-free paths for multiple mobile agents operating in shared, obstacle-dense environments.

Traditional MRPP solutions typically follow either centralized or decentralized approaches. Centralized systems compute globally optimal or near-optimal paths by considering the positions and goals of all agents collectively, but suffer from scalability issues due to the combinatorial explosion of the configuration space as agent numbers grow [1]. In contrast, decentralized approaches assign partial autonomy to agents, enabling independent planning and local communication. While these systems improve scalability and robustness, they can result in suboptimal or conflicting solutions when resolving dynamic interactions in real time [2].

In this context, classical algorithms like A* remain foundational for their balance of optimality and computational efficiency in single-agent applications. However, extending these algorithms to multi-agent systems introduces additional complexities involving coordination, priority negotiation, and real- time conflict resolution [3]. As multi-agent algorithms grow increasingly sophisticated—incorporating factors such as task allocation, scheduling, and distributed decision-making—their internal logic often becomes opaque to human operators, reducing interpretability and trust [4]. This complexity has fueled growing interest in Explainable Artificial Intelligence (XAI), which aims to make complex AI systems more transparent, interpretable, and accountable. XAI is especially vital in safety-critical, human-supervised MRS, where operators must validate, approve, or intervene in automated planning and control processes. Prior studies have shown that providing operators with clear, human- understandable explanations of robot behavior improves system performance, operator trust, and error recovery capability [5] [6]. While existing XAI applications in multi-robot systems have typically addressed isolated subproblems like task allocation [7], scheduling [2], or motion planning [8], recent research emphasizes the need for holistic explanation frameworks capable of addressing the tightly coupled interdependencies in multi- robot decision-making [1]. Robot path decisions are influenced not only by static obstacles and start-goal pairs, but also by task priorities, schedules, and system-level optimization goals. Isolated explanations fall short of capturing this complexity. Motivated by these gaps, our research proposes a novel integration of XAI mechanisms into the A* algorithm for multi-agent path planning. Our system extends the classical A* framework by embedding explanation generation into its decision-making process, enabling it to justify path selections, trade-offs, and conflict resolutions in human-readable form. Specifically, the system produces contrastive natural language explanations answering questions like “Why did agent A take path X instead of path Y?” or “Why was this detour necessary?”, considering task priorities, path costs, obstacles, and

conflicts. We enhance the A* search by integrating a local explanation module that tracks node expansion decisions—including heuristic updates, g-costs, and conflict avoidance—and uses an XAI translation layer to convert this reasoning into natural language. Simulated experiments demonstrate that this explainable framework improves operator comprehension, trust, and intervention capability without compromising efficiency.

- Related Work

In this section, we position our work in relation to prior research in multi-agent planning, explainable AI planning, and explainable task allocation in multi-robot systems. The problem of multi-agent path planning (MAPP) is a central challenge in multi-robot systems, where agents must navigate shared environments without collisions. Classical algorithms like prioritized planning, cooperative A*, and conflict-based search (CBS) perform well in small scenarios but struggle with scalability and real-time demands in larger, heterogeneous teams [9]. Centralized methods such as CBS compute globally consistent paths by iteratively resolving conflicts, though they face exponential computational growth [10]. Decentralized approaches improve scalability by allowing agents to plan locally with inter-agent communication, but often sacrifice global optimality [11]. To enhance performance, researchers have introduced domain-specific heuristics and optimization strategies for dynamic, obstacle-rich environments [12]. Wurm et al. proposed an exploration strategy combining environment segmentation with prioritized planning to minimise conflicts [8]. Recent work also integrates task allocation, scheduling, and motion planning to manage their interdependencies [6]. Neville et al.'s framework, for example, incorporates time-extended task allocation with integrated path planning for scalable, domain-independent coordination [4]. Yet, despite these advances, multi-agent planners often lack explainability, leaving users without insight into path choices, trade-offs, and decision-making processes.

- II. BRIDGING THE GAP: OVERVIEW OF XAI

6.1 Multi-Robot Path Planning

Despite significant progress in multi-robot path planning, most existing approaches have relied on deterministic, optimization-based algorithms like A*, D*, conflict-based search (CBS), and mathematical programming, which are inherently interpretable through transparent decision logic based on cost functions and heuristics. Consequently, there was historically little motivation to adopt explainable artificial intelligence (XAI) in these systems. However, the growing use of machine learning mod- els—including tree-based ensembles and deep neural net- works—to learn heuristics, predict agent behavior, or handle environmental uncertainty, has introduced black-box compo- nents, raising concerns about transparency and trust in safety- critical robotics applications. While SHAP (SHapley Additive exPlanations), rooted in cooperative game theory, offers powerful local and global interpretability [13], its integration into multi-robot planning remains largely unexplored. SHAP has proven valuable in fields like healthcare [14], finance, and critical decision systems [15], where understanding model behavior is essential for trust and compliance. Robotics and path planning have lagged in adopting such explain ability tools due to the complexities of temporal and spatial decision-making. This research proposes a novel integration of SHAP into multi-agent path planning, offering not only performance optimization but also interpretable insights into environmental and agent-specific feature contributions — addressing an emerging need in AI-driven robotics.

- To estimate the complexity of a given multi-robot path planning (MRPP) scenario, we design a synthetic cost function grounded in domain-aware features. This function serves as a proxy for planning difficulty, based on key operational parameters such as distance to goal, inter-agent spacing, task priority, and environment size.

Synthetic Cost Function Formulation

- B Objective:

The synthetic cost aims to encapsulate the estimated "effort" for agents to complete their tasks in a given configuration, without invoking full path planning computations. The model increases cost based on longer traversal distances, tighter agent clustering, lower task urgency, and larger grid environments.

- C Feature-Based Derivation:

We define the synthetic cost using four main components, each reflecting a distinct feature of the task environment.

- Makespan Approximation:

To approximate the time it takes for an agent to reach its destination, we define:

$$\text{Makespan} = \frac{d_{\text{dest}}}{\max(s, 10^{-5})} \tag{6.1}$$

Here, d_{dest} is the Euclidean distance to the goal and s is the agent's speed. The denominator is clamped using $max(s, 10^{-5})$ to avoid division by zero or infinitesimal values, which could threaten the metric.

6.1.1 Collision Penalty

We capture agent-to-agent collision likelihood by computing a penalty based on the average distance among agents:

$$\text{Collision Penalty} = \frac{d_{\text{agents}}}{\max(n, 10^{-5})} \tag{6.2}$$

where d_{dest} is the average pairwise distance between agents, and n is the number of agents. This term increases as agent count grows or spacing decreases.

6.1.2 Priority Factor

A simple inverse scaling reflects task urgency:

$$\text{Priority Factor} = \frac{1}{p} \tag{6.3}$$

where p is the priority of the task. Higher priority values lead to lower penalties, emphasizing urgent tasks.

6.1.3 Grid Factor

To model spatial complexity, we use a logarithmic scaling based on grid size:

$$\text{Grid Factor} = \log(g + 1) \tag{6.4}$$

where g is the grid area or number of cells. The use of $log\,(g + 1)$ ensures the factor increases with grid size while avoiding issues with $g = 0$.

6.1.4 Combined Cost Function

From (6.1) to (6.4) the total synthetic cost is computed as:

$$\text{Cost} = \left(\frac{d_{\text{dest}}}{\max(s, 10^{-5})} + \frac{d_{\text{agents}}}{\max(n, 10^{-5})} \right) \cdot \frac{1}{p} \cdot \log(g + 1) \tag{6.5}$$

This (6.5) ensures that each feature contributes proportionally to the final cost. The additive term inside the parentheses combines the time and interaction penalties, while the multiplicative terms scale the result based on task urgency and spatial complexity.

6.1.5 OVERVIEW of SHAP

SHAP (SHapley Additive exPlanations) is a unified framework for interpreting predictions made by machine learning models. Rooted in cooperative game theory, SHAP attributes the prediction of an instance to its feature contributions by computing **Shapley values** [19]. It guarantees three desirable properties: **local accuracy, missingness**, and **consistency**, making it particularly suitable for tasks where interpretability is crucial, such as optimising synthetic costs in multi-robot path planning.

6.1.5.1 Tree SHAP for Gradient-Boosted Models

For tree-based models like XGBoost or Random Forests, **Tree SHAP** provides an efficient algorithm to compute exact SHAP values with polynomial time complexity $O(T \cdot L \cdot D2)$, where T is the number of trees, L is the number of leaves, and D is the maximum depth of the trees.

This is particularly advantageous in multi-robot path planning, where large feature spaces may emerge from high-dimensional sensor data or environment grids.

6.1.5.2 Theoretical Foundation

SHAP draws upon Shapley values from cooperative game theory, where a "game" consists of features working together to produce a prediction, and the "payout" is the model output. For a model f and a feature iii, the Shapley value ϕ_i is defined as:

$$\phi_i(f) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|! \cdot (|N| - |S| - 1)!}{|N|!} [f(S \cup \{i\}) - f(S)] \tag{6.6}$$

Where:

1. N is the set of all features.
2. S is a subset of features excluding i.
3. $f(S)$ is the model prediction when only features in S are known.
4. i denotes the SHAP value for feature i.

This formulation ensures that each feature is fairly credited for its contribution to the output.

6.1.6 SHAP in Synthetic Cost Optimization

In the context of optimizing a **synthetic cost function** $C_{\text{synthetic}}$ composed of various subcomponents such as **makespan**, **collision penalty**, **priority**, and **grid complexity**, SHAP allows the decomposition of the cost into interpretable contributions:

Let the total synthetic cost be:

$$C_{\text{synthetic}} = f(x_1, x_2, \ldots, x_n) \tag{6.7}$$

Where x_1 are input features such as destination distance d_{dest}, number of agents n, priority index p, and grid complexity g. SHAP yields the decomposition:

$$f(x) = \phi_0 + \sum \phi_n \tag{6.8}$$

Where ϕ_0 is the base value (expected prediction), and ϕ_i is the SHAP value corresponding to feature x_i.

6.1.7 Formulating the SHAP Synthetic Cost

By integrating SHAP values into the cost model, we can define a **SHAP-aware synthetic cost** as:

$$C_{\text{SHAP}} = \sum_{i=1}^{n} w_i \cdot f_i \tag{6.9}$$

Here:

1. f_i are the original cost components such as:
   (a) $f_1 = \frac{d_{\text{dest}}}{\max(s, 10^{-5})}$
   (b) $f_2 = \frac{d_{\text{agents}}}{\max(n, 10^{-5})}$
   (c) $f_3 = \frac{1}{p}$
   (d) $f_4 = \log(g + 1)$
2. w_i are normalized SHAP weights that prioritize features with greater impact.

This reweighted formula can guide cost-aware optimization and decision-making in decentralized robot control or reinforcement learning policies.

$$x = \left(0.3647 \cdot \frac{D_s}{S} + 0.2422 \cdot \frac{D_a}{N} + 0.3118 \cdot \frac{1}{P} + 0.0735 \cdot \ln(G+1) + 0.0078 \cdot T\right) \tag{6.10}$$

6.1.8 Visualization and Interpretation

Using SHAP's visual tools, such as waterfall plots, force plots, and bar plots of mean absolute SHAP values, practitioners can:

- Diagnose why a certain cost was predicted.
- Identify dominant cost drivers.
- Adjust planning parameters dynamically based on feature importance.

6.2 How We Addressed the Problem

To address the lack of interpretability in data-driven multi-robot path planning, we integrated SHAP (SHapley Additive Explanations) into the system to explain the neural network's predictions by quantifying the contribution of each input feature to the estimated path cost. Using SHAP's KernelExplainer [20], we analyzed feature vectors associated with decision points along a robot's path, generating interpretable insights such as "Obstacle at (x, y) increased cost by 0.12" or "Goal proximity reduced cost by 0.25." This feature-wise breakdown offered a transparent view into the rationale behind path decisions, addressing critical challenges in trust, transparency, and accountability in AI-driven robotics.

This integration delivered several key benefits. First, it revealed the most influential features affecting cost predictions, such as nearby obstacles, task priority, or distance to the goal, allowing for model fine-tuning and increased reliability. Second, it enabled the detection of model errors or biases, such as an overreliance on noisy or irrelevant inputs, which led to improvements in feature engineering. Third, SHAP supported adaptive cost tuning, enabling dynamic adjustment of feature weights or heuristic guidance, resulting in more efficient, conflict-free, and smoother paths. Finally, by providing quantitative, human-understandable explanations for decision

points, SHAP enhanced debugging and supported informed model refinement, a capability emphasized as essential in safety-critical applications by recent XAI research [21], [22].

The `synthetic_mapf_cost_function` defined a custom cost metric for multi-agent pathfinding (MAPF), combining key factors influencing agent motion. It incorporated makespan (distance to destination divided by speed), a collision penalty (average distance between agents relative to their count), a priority factor (inversely weighting task priority), and a grid factor (environmental scaling based on grid size logarithm). The total cost was clamped to a minimum of 0.1 to avoid destabilizing near-zero values. This composite cost model balanced environmental, temporal, and agent-specific variables, promoting safe, efficient, and context-aware paths in dynamic environments.

SHAP analysis provided further insights during the path planning process for six agents. The SHAP summary plot showed the average feature contribution to cost predictions. Task priority was the most influential (3.09), followed by speed, distance to destination, inter-agent distance, and number of agents. Less impactful factors included grid size and time. These findings align with recent perspectives in explainable AI research, emphasizing the criticality of feature attribution and model transparency in decision-support systems [23].

Finally, the planner, using time-space A*, successfully avoided potential collisions by dynamically rerouting agents when conflicts were predicted at specific cells and time steps. For example, Agent 2 and Agent 3 avoided a collision at time 11 in cell (3,3). However, Agent 0 failed to find a collision-free path, likely due to congestion or limited space, demonstrating the practical limitations and opportunities for explainable refinements, as supported in prior studies on interpretable AI decision-making in robotics [15], [24].

$$x = \left(0.3647 \cdot \frac{D_s}{S} + 0.2422 \cdot \frac{D_a}{N} + 0.3118 \cdot \frac{1}{P} + 0.0735 \cdot \ln(G+1) + 0.0078 \cdot T\right)$$

6.3 Result

6.3.1 Enhanced Additive Cost Function for MAPF

The additive_mapf_cost_function represents a SHAP-informed refinement of the original synthetic cost function used in multi-agent pathfinding (MAPF). Unlike the earlier multiplicative model, this version adopts an additive approach where each feature's contribution to the total cost is weighted by coefficients derived from SHAP value importance. These weights reflect the actual influence of each feature on the neural network's predictions, bringing a layer of explainability to the cost computation.The function calculates the path cost (costnew) as a weighted sum of normalized input features, including distance to destination over speed,distance between agents over number of agents, inverse of the inverse of the ask priority, logarithmic grid size, and time. By using SHAP- derived coefficients such as 0.3947 for destination distance and 0.4618 for task priority, the model highlights the relative importance of each factor in influencing the path decision. It also includes safeguards like max(..., 1e-5) to prevent division by zero, ensuring numerical stability. The final cost is clamped to a minimum of 0.1 to avoid unrealistic underes- timation. Overall, this SHAP-based additive formula enhances interpretability, data-driven tuning, and trust in the decision- making process for robotic path planning

6.3.2 Cost Function Comparison: Evaluating SHAP-Based Optimization

Fig. 6.1 **compares path planning costs using three methods: the original synthetic cost (blue), SHAP-informed cost (green), and neu-**

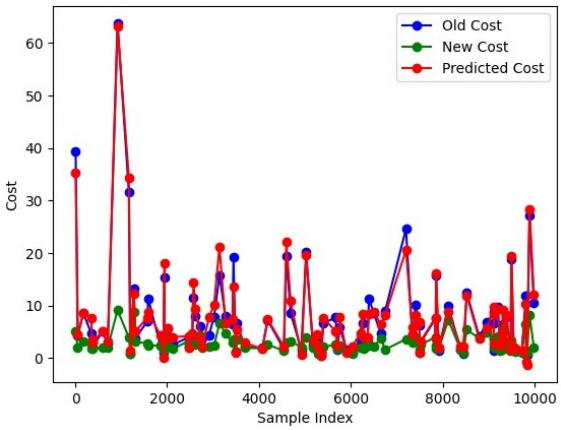

Figure 6.1: Cost Comparison Across A Random Sample

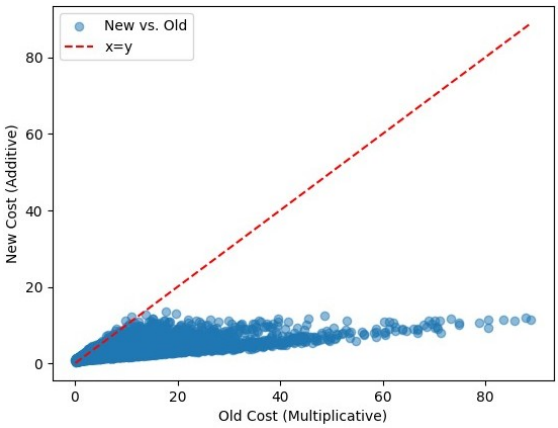

Figure 6.2: old-vs-new-cost

ral network-predicted cost (red). The SHAP-based costs are smoother and consistently lower, indicating improved efficiency. The model's predictions closely follow the original costs, occasionally aligning with the improved SHAP-based estimates.

6.3.3 Old vs. New Cost Function Comparison Summary

Fig. 2 compares the original multiplicative cost function with a SHAP-based additive version. Most points lie below the $y = x$ line, indicating consistently lower, more interpretable costs. While the new function improves explainability and conservativeness, it scales less aggressively in high-cost cases, suggesting possible adjustments for full equivalence. In Multi-Agent Path Finding (MAPF), collision avoidance is handled using Time-Space A*, where each node includes both spatial and temporal

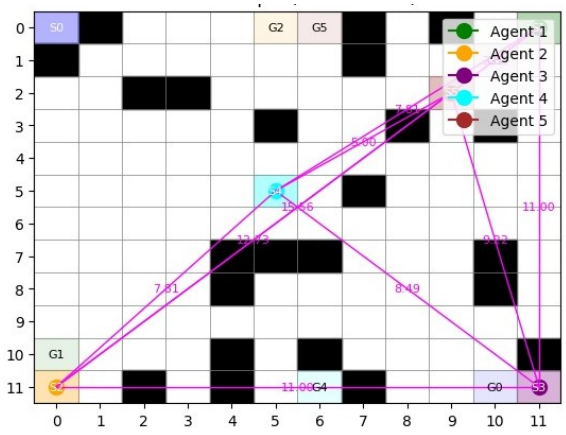

Figure 6.3: Time Step 0 (No Collisions)

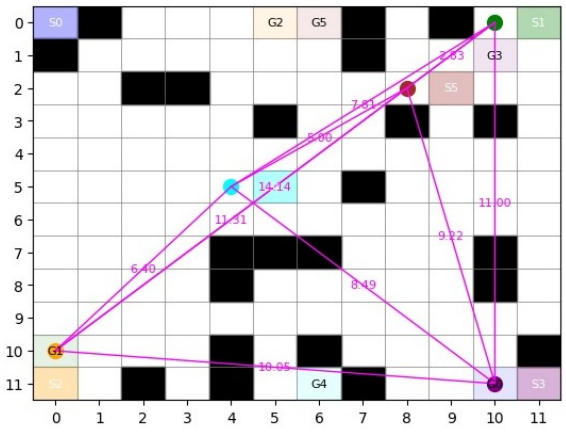

Figure 6.4: time-step-1(No Collisions)

information (x, y, t). As each agent plans its path, a reservation table is used to record which grid cells are occupied at what times. This helps ensure that no two agents occupy the same space at the same time (vertex conflict) or cross paths simultaneously in opposite directions (edge conflict). When such a conflict is detected during path expansion, the algorithm explores alternate routes or wait actions.

Once all agents have their paths, the visualization is done iteratively, simulating each timestep one by one. At each step, the current positions of all agents are plotted on the grid. This allows users to see how agents move across the environment in real-time while avoiding collisions. The agents are typically displayed using different colors or shapes, making it easier to track individual paths.

The plotting is updated in a loop using libraries like matplotlib, where each timestep's frame is drawn, a short delay is added, and the frame is refreshed. This

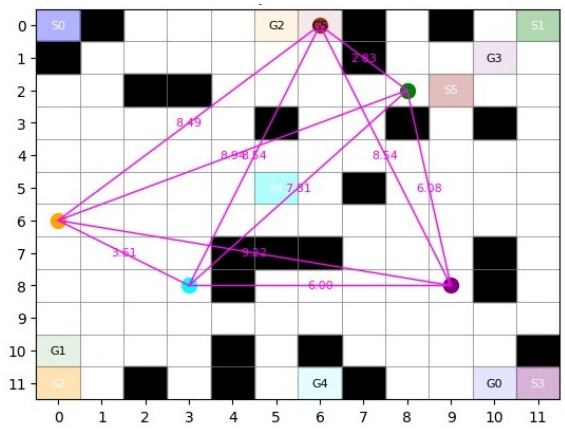

Figure 6.6: time-step-5(No Collisions)

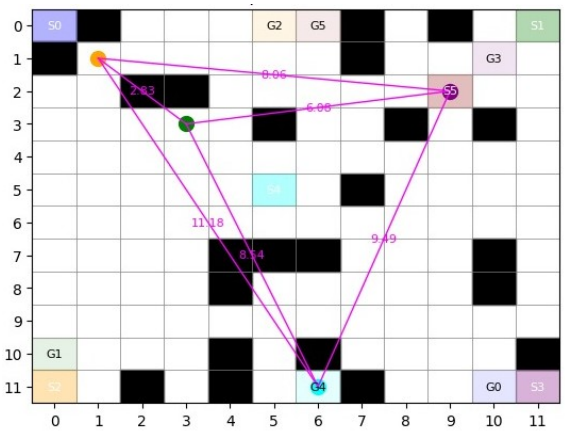

Figure 6.7: time-step-11(No Collisions)

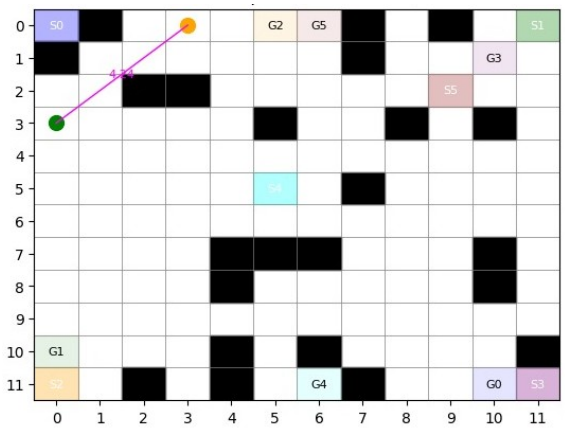

Figure 6.8: time-step-14(No Collisions)

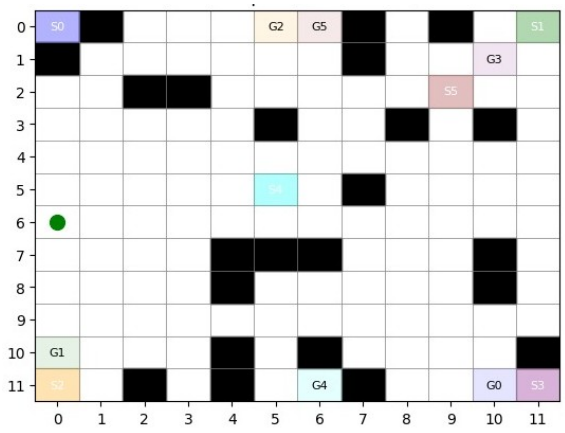

Figure 6.9: time-step-17(No Collisions)

creates a smooth animation of agents navigating the space. The approach not only helps validate the correctness of collision-free paths but also provides a clear visual representation of cooperative path planning.

These images show a multi-agent pathfinding simulation across different time steps (0, 1, and 5). Each agent (represented with colored circles) moves from a unique start location to a goal location on a grid while avoiding obstacles (black cells) and collisions with other agents. The magenta lines indicate the distances between agents and their next positions. Over time, you can see that agents navigate the environment efficiently without collisions, demonstrating successful decentralised or coordinated movement planning.

Part 3

Chapter 7

Conclusion

In this thesis, we proposed a hybrid nature inspired algorithms for robot path planning. The hybrid combination APGO presented in this paper outperformed the other combination presented in the paper as well as it also produced better results than the traditional algorithms when operated individually. The result presented in this paper progressed gradually, first we tested only single algorithms, then we made a combination of two algorithms and finally then we combined the three algorithms. The involvement of GSO algorithm within the combination of ACO and PSO made the node selection behavior faster and effecient. The path length and path time parameters are considered for the validation of results. Both the parameters resulted in a very effecient way. The future extension of this work could be to test this hybrid approach to the dynamic environment where the nature of obstacles are moving and having multiple robots in the environment.

In conclusion, this thesis explored the use of nature-inspired algorithms for robot path planning. The study investigated several algorithms, including ant colony optimization, particle swarm optimization, and genetic algorithms, and evaluated their performance on different robot scenarios. The results showed that nature-inspired algorithms can efficiently and effectively solve robot path planning problems. In particular, ant colony optimization and genetic algorithms performed well in scenarios with complex environments, while particle swarm optimization showed promising results in situations where the robot needs to reach its goal in the shortest time

possible. The iterative improvement of path finding for several robots was given in this paper. The idea employs two MAPF solvers as sub-procedures: an imperfect solver to come up with a preliminary solution and an ideal solver to hone it.The framework discovers a solution with acceptable costs in a given situation, despite the fact that this does not ensure finding the optimal answer quick computation time that is highly scalable. Additionally, it has anytime planning, which is a desirable feature for real-time systems with strict deadlines. The studies show that regardless of the starting solutions, the cost is lowered to almost optimal levels; nonetheless, it is preferable to start with solutions that are at least adequate in terms of efficiency because we may develop better solutions sooner. Therefore, the following whenever MAPF method will be useful. Start numerous initial solvers in parallel with varying runtime and solution quality portfolios . After that, refine the initial answer you came up with. If another initial solver gets a better solution compared to the refined solution at that time, replace the current one with the new one. Moreove,

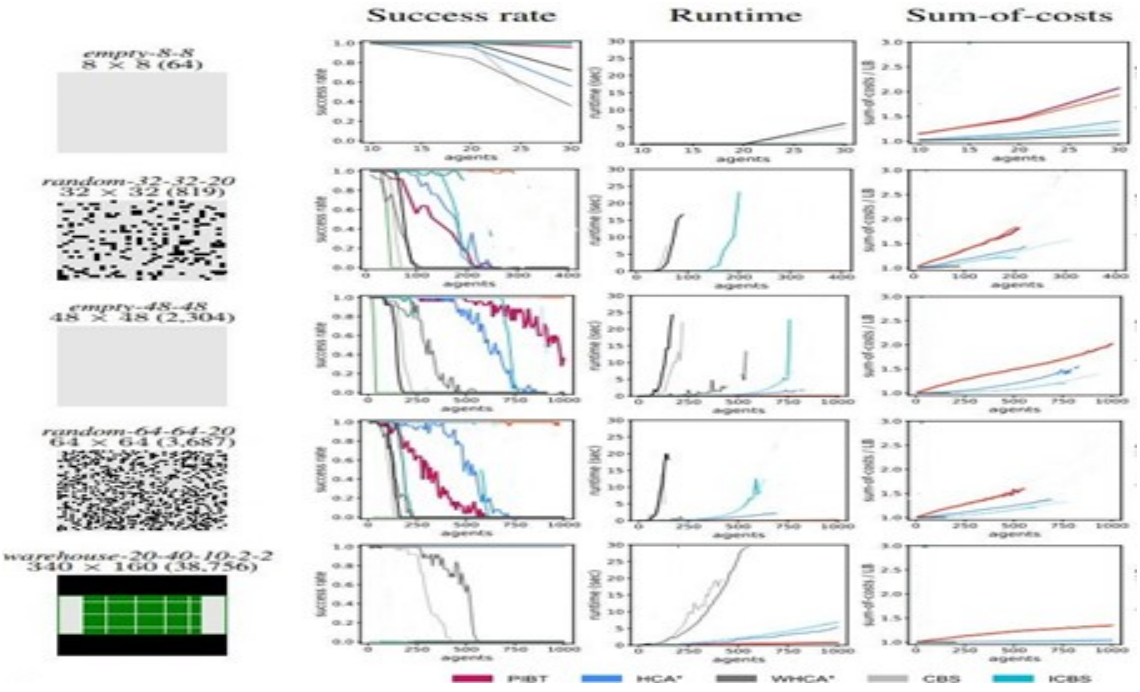

Figure 7.1: Comparison of Various Algorithms

the study highlights the importance of parameter tuning and problem formulation in achieving the best performance of nature-inspired algorithms. The algorithm's parameters, such as the number of ants or particles, play a critical role in achieving

		PIBT$^+$	HCA*	WHCA*	ECBS
random-64-64-20 300 agents	cost (init)	1.219	1.069	1.096	1.037
	cost (last)	1.015	1.015	1.015	1.014
	#success	25	23	15	25
	runtime (ms)	43	201	228	3436
lak307d 300 agents	cost (init)	1.190	1.021	1.155	1.007
	cost (last)	1.003	1.003	1.003	1.003
	#success	25	25	23	25
	runtime (ms)	43	171	247	2306
lak503d 500 agents	cost (init)	1.148	1.021	-	1.026
	cost (last)	1.019	1.018	-	1.018
	#success	25	25	1	24
	runtime (ms)	376	5716	-	54433

Figure 7.2: The Detailed Results with different Initial Solvers

the best results for different scenarios. Problem formulation, such as defining the fitness function, also affects the quality of the solutions. Overall, the results of this study suggest that nature-inspired algorithms can provide an efficient and effective way to solve robot path planning problems. Further research is needed to investigate more complex scenarios and explore different types of algorithms to improve the performance of robot path planning.

7.1 over all conclusion

7.1.1 APGO

The inclusive conclusion regarding robot path planning optimisation using the discussed techniques is that hybrid approaches and the integration of Explainable AI (XAI) offer significant improvements over traditional and individual methods, primarily in terms of path efficiency, interpretability, and cost reduction.

Here's a breakdown Enter Caption of the key conclusions: Effectiveness of Hybrid Nature-Inspired Algorithms (APGO): The research demonstrates the value

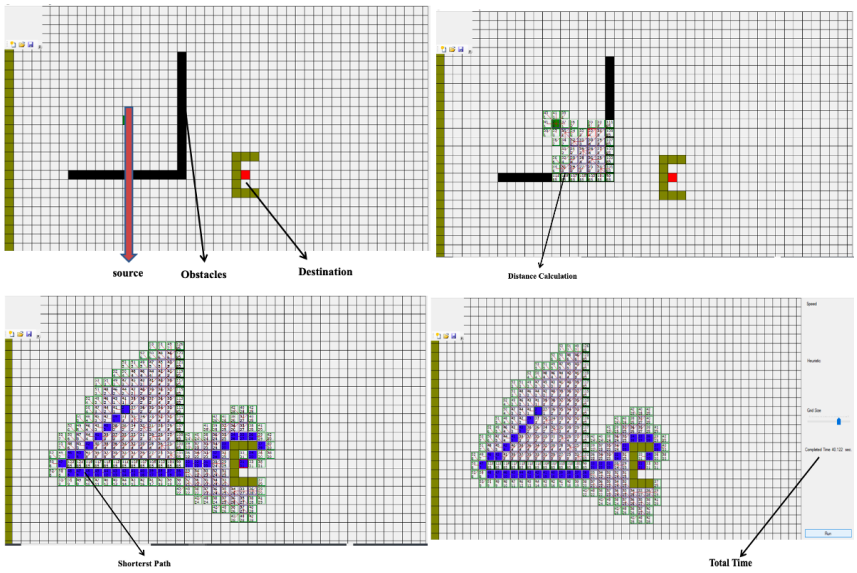

Figure 7.3: Working-APGO

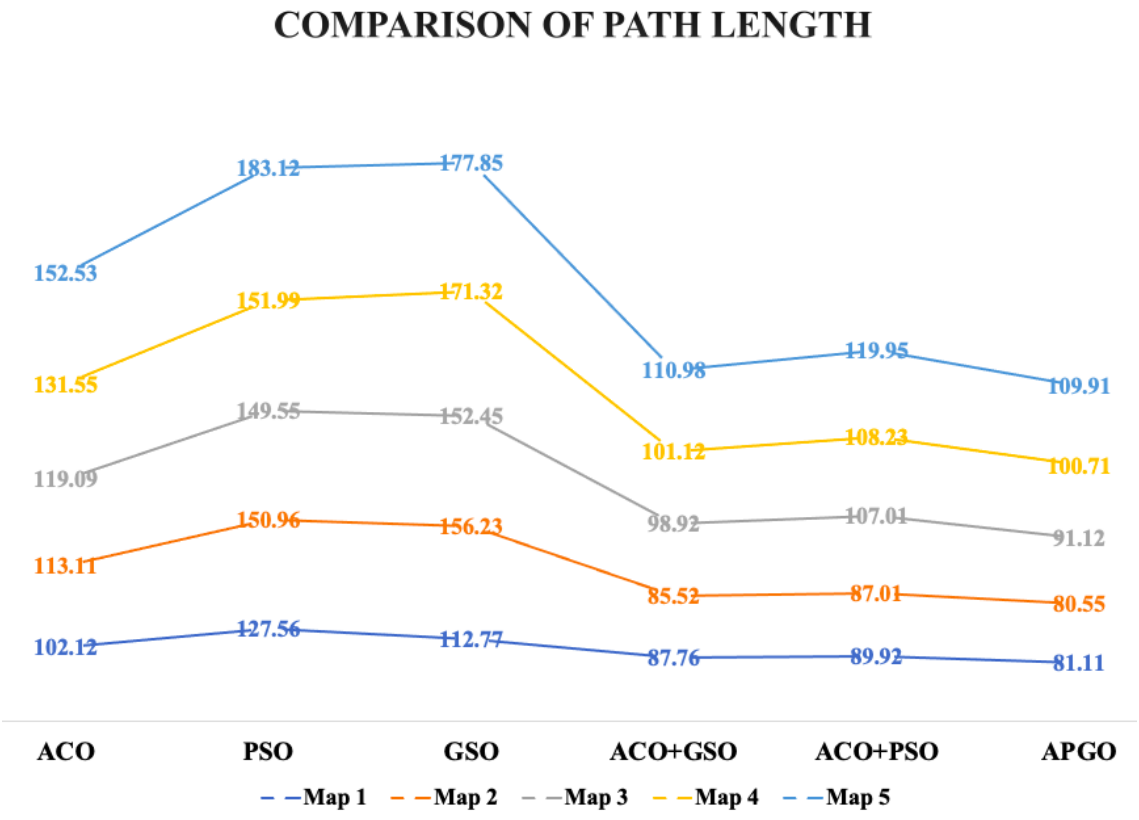

Figure 7.4: Comparison of Path Length

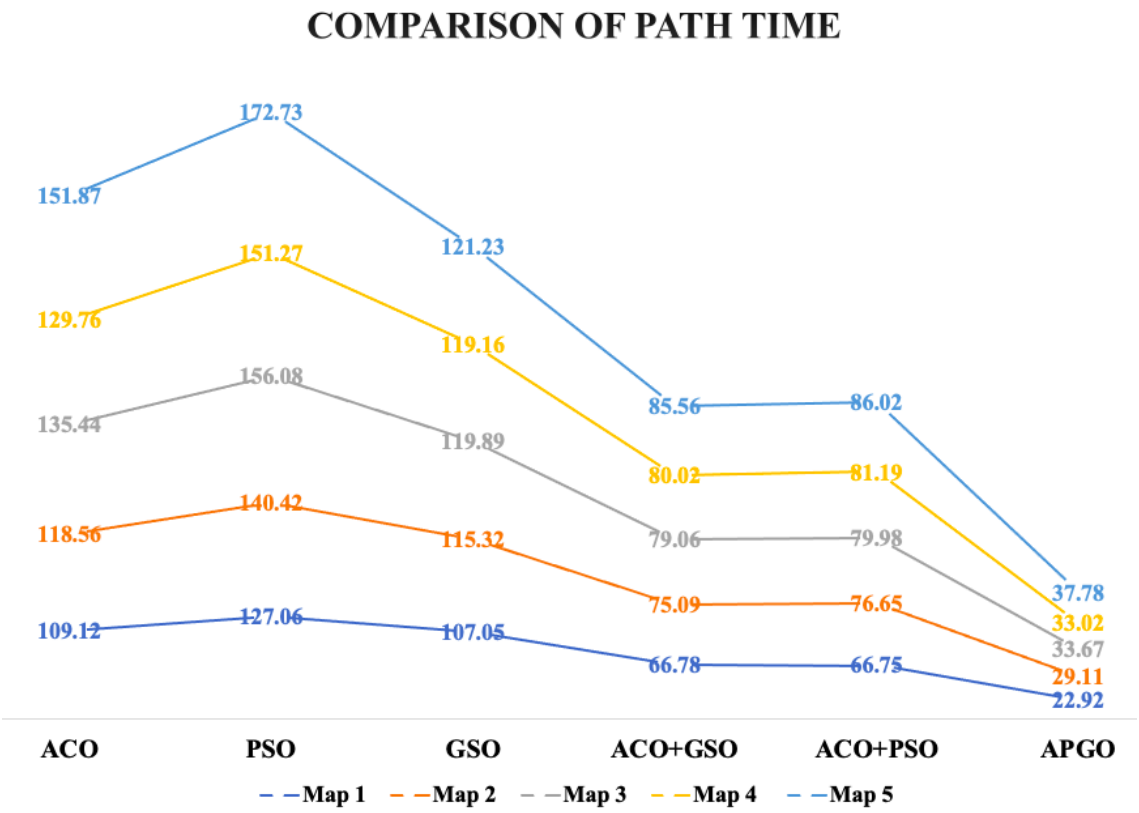

Figure 7.5: Comparison of Path Time

of combining nature-inspired algorithms like Ant Colony Optimisation (ACO), Particle Swarm Optimisation (PSO), and Glow-worm Swarm Optimisation (GSO) for robot path planning. The proposed hybrid algorithm, termed APGO (ACO, PSO, GSO), was developed by gradually combining algorithms after testing them individually and in pairs. The intention was to imbibe the benefits of each optimisation technique. The core conclusion regarding APGO is that it outperformed both the individual ACO, PSO, and GSO algorithms and the pairwise hybrid combinations (ACO+GSO, ACO+PSO),

- This superior performance was validated through experiments across multiple maps, showing significantly lower path length and shorter path time compared to the other methods.
- Specifically, the involvement of the GSO algorithm in the ACO-PSO combination made the node selection behavior faster and more efficient.

The APGO hybrid effectively performs both exploratory and exploitative search, which is noted as an advantage over traditional approaches.

- The results indicate a great improvement in terms of performance for this specific problem using APGO,

Benefits of Integrating Explainable AI (XAI):

Comprehend the "black-box" nature introduced by machine learning components in robotics, XAI (specifically SHAP) was integrated into multirobot path planning to provide transparency and trust, especially crucial in safety-critical applications,

- The combination made it easier to explain and model things. SHAP explained neural network predictions by quantifying the contribution of each input feature to the estimated path cost, providing human-understandable insights into path decisions.
- This analysis revealed the most influential features that affect cost predictions, such as task priority (identified as the most influential), speed, distance to destination, and inter-agent distance.

XAI integration helped detect model errors or biases, support adaptive tuning, and improve debugging by providing quantitative explanations for decision points.

- A direct and tangible result of the XAI analysis was the development and evaluation of a SHAP-informed additive cost function.
- This new SHAP-based additive formula demonstrated a significantly lower average cost (2.9692 vs 4.4864), representing an average reduction of approximately 33.82 compared to the older multiplicative formula.SHAP-based costs were found to be smoother and consistently lower, suggesting improved efficiency in the cost metric itself, The system within which XAI was integrated (utilizing Time-Space A*) also successfully demonstrated collision avoidance through dynamic rerouting.

7.1.2 Overall Contribution and Future Direction:

- The body of work, encompassing both the APGO hybrid and the XAI-informed approach, highlights successful strategies for addressing the complex challenges of robot path planning, particularly in multi-robot and potentially dynamic environments.
- The exploration of different algorithms (traditional, heuristic, metaheuristic, and their combinations) shows a progression towards more efficient and robust solutions.
- Future work is identified, particularly extending the APGO hybrid approach to dynamic environments with moving obstacles and multiple robots, and continuing research into more complex scenarios and different algorithms. The MAPF section also suggests iterative refinement techniques to facilitate the rapid discovery and refinement of initial solutions.
- In essence, the conclusion across the sources is that sophisticated hybrid algorithms like APGO are highly effective for optimising fundamental path planning metrics, while integrating XAI techniques like SHAP provides crucial benefits in terms of the rapid identification and enhancement of preliminary solutions. and refining the underlying cost functions and decision-making processes in complex robotic systems.

XAI-conclusion: Across the dataset, the average cost computed using the old (multiplicative) formula is 4.4864, which serves as a baseline for comparison. The new (additive) formula, on the other hand, yields an average cost of 2.9692, which is significantly lower. This means that on average, the new formula assigns lower costs to the scenarios, with a reduction of approximately 33.82% compared to the old formula. This percentage difference reflects a consistent trend throughout the data set, indicating that the new cost function is about a third lower on average. There is a smaller cost reduction for a single sample scenario with the new formula, approximately 3.43% lower than with the previous one. Additionally, a neural network trained to predict the old (multiplicative) cost outputs a predicted cost of

3.1854 for this specific scenario. This predicted value lies between the old and new costs, as expected, since the model was trained on old cost labels.Overall, this analysis shows that the new cost function tends to produce systematically lower values, and the neural network approximates the old formula reasonably well.

Bibliography

[1] L. Hu, Y. Jiang, F. Wang, K. Hwang, M. S. Hossain, and G. Muhammad, "Follow me robot-mind: Cloud brain based personalized robot service with migration," *Future Gener. Comput. Syst.*, vol. 107, pp. 324–332, 2020. [Online]. Available: https://doi.org/10.1016/j.future.2020.01.041

[2] B. Liu, L. Wang, and M. Liu, "Lifelong federated reinforcement learning: A learning architecture for navigation in cloud robotic systems," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4555–4562, 2019.

[3] X. Ma and Y. Huang, "Research on mobile cloud robotics based on cloud computing," pp. 807–810, 2016/01. [Online]. Available: https://doi.org/10.2991/ifmeita-16.2016.148

[4] A. Koubaa, M. Alajlan, and B. Qureshi, "Roslink: Bridging ros with the internet-of-things for cloud robotics," pp. 265–283, 2017.

[5] S. Chinchali, A. Sharma, J. Harrison, A. Elhafsi, D. Kang, E. Pergament, E. Cidon, S. Katti, and M. Pavone, "Network offloading policies for cloud robotics: a learning-based approach," *arXiv preprint arXiv:1902.05703*, 2019.

[6] J. Kuffner, "Cloud-enabled humanoid robots," 2010.

[7] Y. Chen and H. Hu, "Internet of intelligent things and robot as a service," *Simulation Modelling Practice and Theory*, vol. 34, pp. 159–171, 2013.

[8] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," pp. 2786–2793, 2017.

[9] G. Loukas, T. Vuong, R. Heartfield, G. Sakellari, Y. Yoon, and D. Gan, "Cloud-based cyber-physical intrusion detection for vehicles using deep learning," *Ieee Access*, vol. 6, pp. 3491–3508, 2017.

[10] A. Botta, L. Gallo, and G. Ventre, "Cloud, fog, and dew robotics: Architectures for next generation applications," pp. 16–23, 2019.

[11] E. Cardarelli, V. Digani, L. Sabattini, C. Secchi, and C. Fantuzzi, "Cooperative cloud robotics architecture for the coordination of multi-agv systems in industrial warehouses," *Mechatronics*, vol. 45, pp. 1–13, 2017.

[12] A. Iosup, R. Prodan, and D. Epema, "Iaas cloud benchmarking: approaches, challenges, and experience," pp. 83–104, 2014.

[13] J. Wan, S. Tang, Q. Hua, D. Li, C. Liu, and J. Lloret, "Context-aware cloud robotics for material handling in cognitive industrial internet of things," *IEEE Internet of Things Journal*, vol. 5, no. 4, pp. 2272–2281, 2017.

[14] H. Cao, R. Chen, Y. Gu, and H. Xu, "Cloud-assisted tracking medical mobile robot for indoor elderly," pp. 927–930, 2017.

[15] Z. Du, L. He, Y. Chen, Y. Xiao, P. Gao, and T. Wang, "Robot cloud: Bridging the power of robotics and cloud computing," *Future Generation Computer Systems*, vol. 74, pp. 337–348, 2017.

[16] Y. Chen, Z. Du, and M. García-Acosta, "Robot as a service in cloud computing," pp. 151–158, 2010.

[17] L. P. E. Toh, A. Causo, P.-W. Tzuo, I.-M. Chen, and S. H. Yeo, "A review on the use of robots in education and young children," *Journal of Educational Technology & Society*, vol. 19, no. 2, pp. 148–163, 2016.

[18] S. Volkov and O. Sukhoroslov, "Simplifying the use of clouds for scientific computing with everest," *Procedia computer science*, vol. 119, pp. 112–120, 2017.

[19] G. A. Pereira, A. K. Das, V. Kumar, and M. F. Campos, "Decentralised motion planning for multiple robots subject to sensing and communication constraints," in *Proceedings of the Second Multi-Robot Systems Workshop*, 2003, pp. 267–278.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[21] D. Gunning, M. Stefik, J. Choi, T. Miller, S. Stumpf, and G.-Z. Yang, "Xai—explainable artificial intelligence," *Science robotics*, vol. 4, no. 37, p. eaay7120, 2019.

[22] J. H. Jensen, K. P. B. Sørensen, J. l. F. Sejersen, and A. Sarabakha, "Multi-agent path planning in complex environments using gaussian belief propagation with global path finding," *arXiv preprint arXiv:2502.20369*, 2025.

[23] A. M. Salih, Z. Raisi-Estabragh, I. B. Galazzo, P. Radeva, S. E. Petersen, K. Lekadir, and G. Menegaz, "A perspective on explainable artificial intelligence methods: Shap and lime," *Advanced Intelligent Systems*, vol. 7, no. 1, p. 2400304, 2025.

[24] G. Van den Broeck, A. Lykov, M. Schleich, and D. Suciu, "On the tractability of shap explanations," *Journal of Artificial Intelligence Research*, vol. 74, pp. 851–886, 2022.

Bibliography

[1] L. Hu, Y. Jiang, F. Wang, K. Hwang, M. S. Hossain, and G. Muhammad, "Follow me robot-mind: Cloud brain based personalized robot service with migration," *Future Gener. Comput. Syst.*, vol. 107, pp. 324–332, 2020. [Online]. Available: https://doi.org/10.1016/j.future.2020.01.041

[2] B. Liu, L. Wang, and M. Liu, "Lifelong federated reinforcement learning: A learning architecture for navigation in cloud robotic systems," *IEEE Robotics and Automation Letters*, vol. 4, no. 4, pp. 4555–4562, 2019.

[3] X. Ma and Y. Huang, "Research on mobile cloud robotics based on cloud computing," pp. 807–810, 2016/01. [Online]. Available: https://doi.org/10.2991/ifmeita-16.2016.148

[4] A. Koubaa, M. Alajlan, and B. Qureshi, "Roslink: Bridging ros with the internet-of-things for cloud robotics," pp. 265–283, 2017.

[5] S. Chinchali, A. Sharma, J. Harrison, A. Elhafsi, D. Kang, E. Pergament, E. Cidon, S. Katti, and M. Pavone, "Network offloading policies for cloud robotics: a learning-based approach," *arXiv preprint arXiv:1902.05703*, 2019.

[6] J. Kuffner, "Cloud-enabled humanoid robots," 2010.

[7] Y. Chen and H. Hu, "Internet of intelligent things and robot as a service," *Simulation Modelling Practice and Theory*, vol. 34, pp. 159–171, 2013.

[8] C. Finn and S. Levine, "Deep visual foresight for planning robot motion," pp. 2786–2793, 2017.

[9] G. Loukas, T. Vuong, R. Heartfield, G. Sakellari, Y. Yoon, and D. Gan, "Cloud-based cyber-physical intrusion detection for vehicles using deep learning," *Ieee Access*, vol. 6, pp. 3491–3508, 2017.

[10] A. Botta, L. Gallo, and G. Ventre, "Cloud, fog, and dew robotics: Architectures for next generation applications," pp. 16–23, 2019.

[11] E. Cardarelli, V. Digani, L. Sabattini, C. Secchi, and C. Fantuzzi, "Cooperative cloud robotics architecture for the coordination of multi-agv systems in industrial warehouses," *Mechatronics*, vol. 45, pp. 1–13, 2017.

[12] A. Iosup, R. Prodan, and D. Epema, "Iaas cloud benchmarking: approaches, challenges, and experience," pp. 83–104, 2014.

[13] J. Wan, S. Tang, Q. Hua, D. Li, C. Liu, and J. Lloret, "Context-aware cloud robotics for material handling in cognitive industrial internet of things," *IEEE Internet of Things Journal*, vol. 5, no. 4, pp. 2272–2281, 2017.

[14] H. Cao, R. Chen, Y. Gu, and H. Xu, "Cloud-assisted tracking medical mobile robot for indoor elderly," pp. 927–930, 2017.

[15] Z. Du, L. He, Y. Chen, Y. Xiao, P. Gao, and T. Wang, "Robot cloud: Bridging the power of robotics and cloud computing," *Future Generation Computer Systems*, vol. 74, pp. 337–348, 2017.

[16] Y. Chen, Z. Du, and M. García-Acosta, "Robot as a service in cloud computing," pp. 151–158, 2010.

[17] L. P. E. Toh, A. Causo, P.-W. Tzuo, I.-M. Chen, and S. H. Yeo, "A review on the use of robots in education and young children," *Journal of Educational Technology & Society*, vol. 19, no. 2, pp. 148–163, 2016.

[18] S. Volkov and O. Sukhoroslov, "Simplifying the use of clouds for scientific computing with everest," *Procedia computer science*, vol. 119, pp. 112–120, 2017.

[19] G. A. Pereira, A. K. Das, V. Kumar, and M. F. Campos, "Decentralised motion planning for multiple robots subject to sensing and communication constraints," in *Proceedings of the Second Multi-Robot Systems Workshop*, 2003, pp. 267–278.

[20] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," *Advances in neural information processing systems*, vol. 30, 2017.

[21] D. Gunning, M. Stefik, J. Choi, T. Miller, S. Stumpf, and G.-Z. Yang, "Xai—explainable artificial intelligence," *Science robotics*, vol. 4, no. 37, p. eaay7120, 2019.

[22] J. H. Jensen, K. P. B. Sørensen, J. l. F. Sejersen, and A. Sarabakha, "Multi-agent path planning in complex environments using gaussian belief propagation with global path finding," *arXiv preprint arXiv:2502.20369*, 2025.

[23] A. M. Salih, Z. Raisi-Estabragh, I. B. Galazzo, P. Radeva, S. E. Petersen, K. Lekadir, and G. Menegaz, "A perspective on explainable artificial intelligence methods: Shap and lime," *Advanced Intelligent Systems*, vol. 7, no. 1, p. 2400304, 2025.

[24] G. Van den Broeck, A. Lykov, M. Schleich, and D. Suciu, "On the tractability of shap explanations," *Journal of Artificial Intelligence Research*, vol. 74, pp. 851–886, 2022.